IMPAX 6.2

Desktop Integration Guide

Integrating Applications with the IMPAX Client

AGFA

see more | do more

Copyright information

© Copyright 2005, Agfa-Gevaert N.V., B-2640, Mortsel, Belgium. All rights reserved. No parts of this document may be reproduced, copied, translated, adapted or transmitted in any form or by any means without prior written permission of Agfa-Gevaert N.V.

Trademark credits

Agfa, the Agfa rhombus, and IMPAX are trademarks or registered trademarks of Agfa-Gevaert N.V., Belgium or its affiliates. All other trademarks are held by their respective owners and are used in an editorial fashion with no intention of infringement.

Documentation warranty statement

Characteristics of the products described in this publication can be changed at any time without notice.

The information contained in this document is subject to change without notice. Agfa-Gevaert N.V. makes no warranties or representations, express, implied or statutory, with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose.

Agfa-Gevaert N.V. and its affiliates shall under no circumstances be liable for any damage arising from the use or inability to use any information, apparatus, method or process described in this document. Agfa-Gevaert N.V. and its affiliates shall not be liable for errors contained herein or for incidental or consequential damage in connection with the furnishing, performance, or use of this manual.

2006-11-19

Revision record

Document Name: IMPAX 6.2 Desktop Integration Guide

Revision Date	Description
November 2006	Released for publication.

Additional documentation

This guide is intended for developers who are integrating the IMPAX 6.2 Client with another application. For more information about the IMPAX 6.2 Client, refer to the *IMPAX 6.0 Client Knowledge Base*.

To open the IMPAX 6.2 Client Knowledge Base

1. Ensure the documentation has been installed on the Application Server the Client connects to.
2. Launch the IMPAX Client and do one of the following:
   - Press **F1**.
   - From the List or Configure area bar, click **Help**.

- From the Image area context or top toolbar, click **Help**. 

or

Open a browser window and navigate to **https://<app_server_name>/impax/documents/client/default.htm**.

Contents

1 Getting Started 7

Prerequisite knowledge 7
What is context sharing? 7
About the IMPAX Context Server 7
Context Server architecture 9
Enabling Context Server integrations 9
IMPAX 6.x and customized automation 9
New in IMPAX 6.2 9

2 Logging into IMPAX 12

Default Context Server behavior at startup 12
Setting up single sign-on 12
Setting up single sign-off 17

3 Selecting Studies for Display 19

Identifying the studies to be displayed 19
Adding studies to an IMPAX worklist 20
Cycling studies into the Image area 21

4 Displaying Study Images 23

Clearing the Image area 23
Navigating to a study in the Image area 24

5 Dictating Studies 26

Launching a dictation application from IMPAX 26
Understanding dictation status 26
Dictating from the IMPAX Client 27
Dictating from an integrated application 30

6 Creating Scripts 33

Scripting basics 33
Writing scripts 33
Script examples 34

7 DoAction() Methods and Action() Events 36

ADD_TO_DISPLAY_LIST 36
ADD_TO_DISPLAY_LIST_WITH_PRIORS 37
BLANK_DISPLAY 38

DICTATION_STARTED 38
DICTATION_COMPLETED 39
GETSTUDIESINDISPLAY 39
GETUIDBYPICACN and GETUIDSBYPID 40
HIDE_SELECT and SHOW_SELECT 40
LOGOUT from Integrated Application 41
LOGOUT from IMPAX Client application 41
REMOVE_ALL_FROM_CYCLE_LIST 42
REMOVE_ALL_FROM_WORKLIST 42
ROLLBACK_IN_PROGRESS_STUDY 43
RUN_SCRIPT 43
SET_FOCUS 44
SET_WINDOW_STATE 44
STUDY_CHANGED 44
SYNC_DISPLAY 45
UPDATE_STUDY_STATUS 46
..... 47

8 Events Reference 48

AccessionNumberChange 48
Action 48
AddToDisplayListTrigger 48
AlwaysOnTopChange 49
ApplicationNameChange 49
FontChange 49
IconSizeChange 50
LaunchApplicationTrigger 51
PasswordChange 51
PatientIdentifierChange 51
PerformUserValidationTrigger 52
ResetDisplayListTrigger 57
ScreenAttributeChange 58
ShowDisplayListTrigger 59
StudyIdentifierChange 59
StudyReadTrigger 59
StudyStatusChange 60
UserNameChange 60

9 Methods Reference 61

AddToDisplayList 61
DoAction 61
ALaunchApplication 62
PerformUserValidation 62
ResetDisplayList 62
ShowDisplayList 63
StudyRead 63

10 Properties Reference 64

AccessionNumber 64
AlwaysOnTopFlag 64

ApplicationName 65
BrokerDSN 65
ContextServer 66
FontName 66
FontSize 67
IconSize 67
NumberOfXMonitors 68
NumberOfYMonitors 68
Password 68
PatientId 69
ScreenToCentreOn 69
StudyId 70
StudyStatus 70
UserName 71

Appendix A: IMPAX 6.2 Script Interfaces 73

IScriptApplication 73
IScriptDisplay 77
IScriptListArea 81
Sample script 91

Index 93

Getting Started

1

This chapter gives an overview of IMPAX 6.2 desktop integration using the Context Server. It outlines the prerequisite knowledge, gives an overview of context sharing in general and the IMPAX Context Server in particular, and summarizes what is new in IMPAX 6.2.

Prerequisite knowledge

This guide assumes that the reader has an understanding of Windows COM/ActiveX or .NET programming. An explanation of COM/ActiveX and .NET technology is beyond the scope of this guide.

What is context sharing?

Context is information that is part of an application's operating environment and that can be sensed by the application." That is, a context-aware application can sense changes in the context, without direct user intervention, and change its behavior accordingly. Context information typically includes, but is not limited to, location, identity, and activity of an object.

The IMPAX dictation interlock is an example of context sharing. When a user begins dictating study *A* by clicking the Dictate button in the IMPAX Client application, other users are prevented from dictating the study on other Client stations. The IMPAX Client applications are "aware" that study *A* is currently being dictated by another user.

Context sharing can be extended to include other applications by creating a sufficiently generic "Context Server". The Context Server then acts as a central authority for the operating environment's context information. By sharing context this way, various applications can present users with a discreet, integrated application environment.

Basic context sharing technology is well-suited to integrations that are more discreet; however, it does not scale well to more elaborate workflow integration solutions. Context interpretation becomes increasingly more difficult as the complexity of integration increases. Nevertheless, context sharing provides a simple, robust, and effective means to integrate two or three applications.

About the IMPAX Context Server

The IMPAX Context Server acts as a central reference point for integrating IMPAX with other client desktop applications. However, the IMPAX Context Server goes beyond simple context

sharing to provide a messaging mechanism. This messaging capability enables the Context Server to mitigate the limitations of simple context sharing and to provide for more elaborate integration possibilities.

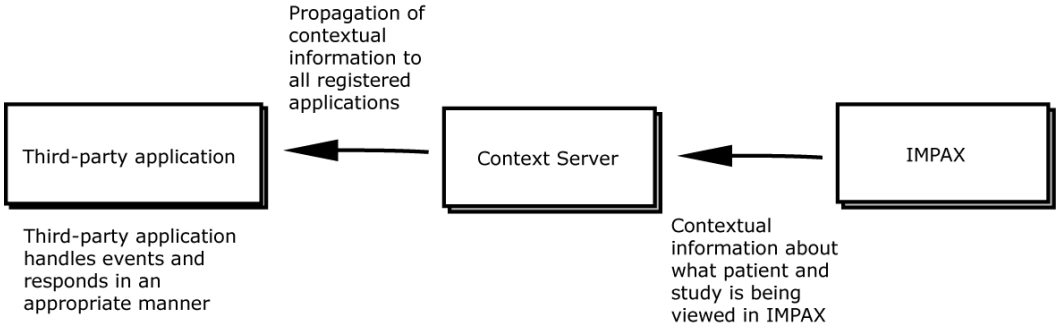
The Context Server is a small Windows 32-bit application (COM executable) that resides on a workstation with the IMPAX Client and other client applications. Client applications register interest in the Context Server to indicate their participation in communication. Participants may then listen for specific events, fire events of their own, respond to context changes, or set context.

Note:

IMPAX context sharing is limited to desktop integration. It connects the IMPAX Client to third-party applications. It cannot be used for back-end connectivity.

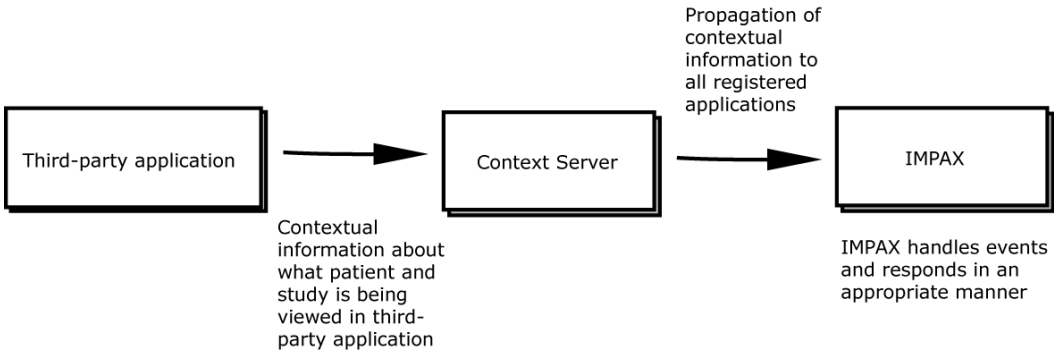
Context sharing is bidirectional. IMPAX can pass context-specific information to the Context Server via its public interface. (See an example in the following section.) The Context Server is responsible for propagating messages by raising events in all the registered applications.

IMPAX passing context to third-party application



Third-party applications can also pass context-specific information to the Context Server via its public interface. (See an example in the following section.) In other words, changes in the context of one application can drive the other application.

Third-party application passing context in IMPAX



Applications may also use the Context Server as a content-independent messaging service. This enables third-party applications and IMPAX to exchange custom messages, extending integrations beyond the Context Server's public interface. Integrations can therefore range from simple context sharing to elaborate workflow integrations.

Context Server architecture

The IMPAX Context Server is an ActiveX executable. It is also a *singleton* component, meaning that only one instance of the Context Server is ever running on the computer. Multiple client applications may be running, but they would all connect to the same instance of the Context Server.

This singleton architecture is managed using two public classes of the IMPAX Context Server: a connector class and a context server class. Client applications may not create an instance of the context server class directly. They must create an instance of the connector class, then obtain a reference to the context server class from the connector class. After obtaining a reference to the context server class, client applications may use the services of the IMPAX Context Server.

Enabling Context Server integrations

To allow IMPAX users to access Context Server integrations, they must be placed in a role that includes a permission with the Enable Context Server Integrations operation selected. For instructions on assigning permissions, refer to “Defining Permissions” (article ID 009628) in the *IMPAX 6.0 Client Knowledge Base*.

IMPAX 6.x and customized automation

The IMPAX 6.x Client uses a new scripting language to provide automation. You can write a VB.NET program to control the behavior of the IMPAX Client and access the objects it contains. You can therefore automate and extend the application in a highly customized manner.

VB.NET scripts are executed in the IMPAX Client using the Visual Studio for Applications (VSA) runtime engine, which is part of the Microsoft .NET Framework. Script code is compiled in memory and executed. At runtime, the script is executed as a full VB.NET application, with access to the IMPAX Client application through a set of custom objects. For more information about and examples of the script interface, refer to *Chapter 6: Creating Scripts* on page 33 and *Appendix A: IMPAX 6.2 Script Interfaces* on page 73.

Finally, in IMPAX 6.x, Client computers no longer have direct ODBC connections to RIS Gateway or PACS broker. As a result, the *BrokerDSN* property, page 65, can no longer be used to look up the Study UID when only the PatientID and AccessionNumber are available. (Instead, use the GETUIDBYPIDACN *DoAction* method, page 62, to achieve this result.)

New in IMPAX 6.2

IMPAX 4.5, 5.*x*, and 6.x Clients all use the same version of the IMPAX Context Server. However, the response to certain events, methods, and properties differs in IMPAX 6.2 from past releases.

New and modified methods and events

The following arguments to the DoAction method can be called from the Integrated Applications such as a RIS:

- **SYNC_DISPLAY**—bring forward a study that is already loaded in the Image area but which is not necessarily visible yet.

- **LOGOUT**—log out of the IMPAX Client.
- **ADD_TO_DISPLAY_LIST**—add a study to the cycle list silently.
- **ADD_TO_DISPLAY_LIST_WITH_PRIORS**—add a study with its relevant priors to the cycle list silently.
- **SYNC_CONTEXT**—fill out the Context Server study identifiers with what is currently visible in the Image area.
- **GETSTUDIESINDISPLAY**—return all studies that are currently visible in the Image area, as well as all other studies that are loaded in the cycle list and have been set aside.
- **BLANK_DISPLAY**—clear the Image area, but leave the cycle list and set aside studies untouched.
- **REMOVE_ALL_FROM_WORKLIST**—remove all studies from the cycle list and set aside studies, clearing the Image Area, but leaving the Advanced Search list untouched.
- **REMOVE_ALL_FROM_CYCLE_LIST**—remove all studies from the cycle list, but leave the Advanced Search list untouched.

Defining new parameters and statuses

The following parameters and statuses have been added in IMPAX 6.2.

KVPid

A KVPid is an identifier, contained in a single string, for one or more studies using KVP's. A KVP is defined as `<keyword>=<value>`, for example `study_uid=1.12.1231`. All KVP's within a KVPid are concatenated using an ampersand (&). The ordering of KVP's within the KVPid is irrelevant.

Each KVPid contains the following four keywords:

- **patient_id**—the Patient ID of the study
- **accession_number**—the Accession Number
- **study_uid**—the DICOM Study Instance UID of the study
- **status**—the status of the study in IMPAX. Possible statuses include:
    - **N**—New
    - **d**—Dictation in progress
    - **t**—Trainee dictation in progress
    - **D**—Dictated
    - **T**—Trainee dictated
    - **r**—Reported
    - **R**—Approved

Each value of a KVP can contain multiple sub values, separated by a backslash (\). In a KVPid, each KVP value must contain the same number of sub values, and sub values on the same relative location within each KVP value must be related to the same study.

KVPid								
KVP_1			&	...	&	KVP_4		
<keyword$_1$>	=	<value$_1$>	&	...	&	<keyword$_4$>	=	<value$_4$>

<keyword$_1$>	=	<subvalue$_{11}$>\...\subvalue$_{1n}$>	&	...	&	<keyword$_4$>	=	<subvalue$_{41}$>\...\subvalue$_{4n}$>
Example: patient_id=41\60&accession_number=85552\94245&study_uid=1.2.124.113532.128.5.1.74.20011230.102134.557173\1.2.124.113532.128.5.1.74.20020129.191658.586884&status=N\D								

When a DoAction method is called by the Integrated Application, a KVPid is constructed by the IMPAX Client to be used in an Action() event, and contains as many values as can be retrieved by IMPAX.

- If the Integrated Application requests to add a study to the cycle list through specifying the accession number only, the resulting Action("STATUS") event contains a KVPid for that study, and all KVP's are filled out with information from the IMPAX database.
    - Integrated Application: `DoAction("ADD_TO_DISPLAY_LIST", "", "12", "")`
    - IMPAX Client: `Action("STATUS", "ADD_TO_DISPLAY", "patient_id=1&accession_number=12&study_uid=1.12.1231&status=N", "0")`
- If a certain value for a KVP can not be retrieved from the IMPAX database, because the referenced study does not exist in IMPAX for example, the value is empty. An empty KVPid is defined as `patient_id=&accession_number=&study_uid=&status=`
    - Integrated Application: `DoAction("ADD_TO_DISPLAY_LIST", "", "12", "")`
    - IMPAX Client: `Action("STATUS", "ADD_TO_DISPLAY", "patient_id=&accession_number=12&study_uid=&status=", "1")`

ResultStatus

The following statuses have been added to ResultStatus:

- **ResultStatus=0**—indicates success
- **ResultStatus=1**—indicates a specific failure (depending on the method that was called)
- **ResultStatus=-1**—indicates failure for unknown reasons

When multiple statuses are returned, they are separated by a backslash (\).

Active studies

An active study is a study that is visible in the Image area, and that is currently in focus for a user. This is the study experiencing any user actions such as Window/Level, dictating, etc.

The Image area can have only one active study per monitor.

- When one study per monitor is displayed, each study on each monitor is active.
- When there are two or more studies visible per monitor, only one study per monitor is active. This is indicated by a white rectangle around the study.
- When the same study spans multiple monitors, then there is only one active study.

Visible studies

A visible study is any study displayed in the Image area, regardless of whether or not it is the active study. For example, in a two study/monitor layout, four studies may be visible.

Logging into IMPAX

2

When integrating IMPAX with another application, you generally want to create a link in the startup of both applications, such that a user who is authenticated to one application also gains access to the other. This chapter explains how to set that up. For details on all events, methods, and properties referenced in this chapter, refer to *Chapter 8: Events Reference on page 48*, *Chapter 9: Methods Reference on page 61*, and *Chapter 10: Properties Reference on page 64*.

Default Context Server behavior at startup

When a user logs into the IMPAX Client application, the following properties are set and their associated events triggered.

Property	Value	Associated event
NumberOfXMonitors	Number of monitors displaying the Image area	*ScreenAttributeChange*
NumberOfYMonitors	1	*ScreenAttributeChange*
ScreenToCentreOn	Which monitor displays the List and Text areas	*ScreenAttributeChange*
UserName	Empty string	*UserNameChange*
Password	Empty string	*PasswordChange*

Setting up single sign-on

With single sign-on, a user logging into IMPAX also gains access to the integrated application. This section explains how to use the Context Server to set up single sign-on.

A prerequisite to single sign-on is that the user must be set up in the IMPAX Client and the integrated application with exactly the same, case-sensitive user ID and password. The IMPAX Client must be running with the Login dialog open before IMPAX can respond to a login event from the Context Server.

As noted in *Enabling Context Server integrations* on page 9, access to the Context Server can be restricted through IMPAX permissions. Access to the single sign-on feature can also be restricted through an IMPAX operation called Allow Single Sign-on. If access is provided to

Context Server but not to single sign-on, single sign-on from IMPAX to the integrated application is not possible. (By default, single sign-on access is off.)

Refer to the *IMPAX 6.0 Client Knowledge Base*: Administering IMPAX > Managing roles and users > Defining permissions (Article ID 009628).

Single sign-on from IMPAX to the integrated application

Logging into the IMPAX Client application sets the UserName and (encrypted) Password properties in the Context Server. Subsequently, the PerformUserValidation method is applied, which generates the *PerformUserValidationTrigger* event.

To provide single sign-on from IMPAX to the integrated application, the integrated application must respond to the *PerformUserValidationTrigger* event by fetching the UserName and Password property, decrypting the password, and logging in with that information.

The precise sequence of communication varies based on the initial state of IMPAX and the integrated application, and the user actions performed. You can decide how many of these states to account for in your integration.

State 1: Both applications running

IMPAX Client and the integrated application are both running, with the Login dialogs displayed. The user logs into IMPAX. To trigger login to the integrated application as well, the integrated application must respond as follows:

Stage	IMPAX Client	Context Server	Integrated application
1	Set **UserName** property →		
2		← *UserNameChange* event →	
3	Encrypt the password. Set **Password** property →		
4		← *PasswordChange* event →	
5	**PerformUserValidation** method →		
6		← *PerformUserValidationTrigger* event →	
7			Get the **UserName** property. Get the **Password** property. Decrypt the password. Log into the application.

State 2: User logged into integrated application only

The user is logged into the integrated application, but not into IMPAX Client. This user, or another user, then logs into the IMPAX Client application. To trigger login to the integrated application at this point, the integrated application must respond as follows:

Stage	IMPAX Client	Context Server	Integrated application
1	Set **UserName** property →		
2		← *UserNameChange* event →	
3	Encrypt the password. Set **Password** property →		
4		← *PasswordChange* event →	
5	**PerformUserValidation** method →		
6		← *PerformUserValidationTrigger* event →	
7			Get the **UserName** property and compare with current user. If the same, do nothing. If different, log out the current user. Get the **UserName** property. Get the **Password** property. Decrypt the password. Log into the application.

State 3: Integrated application started after IMPAX Client

The user logs into the IMPAX Client when the integrated application is not running. A user then starts the integrated application.

Stage	IMPAX Client	Context Server	Integrated application
1			Get the **UserName** property. Get the **Password** property. Decrypt the password. Compare with current user. If a registered user, log into the application. If not, show the Login screen.

Single sign-on from the integrated application to the IMPAX Client

To provide two-way communication, such that logging into the integrated application also logs the user into IMPAX, you have to account for a slightly more complex set of actions.

State 1: Both applications running on Login screen

IMPAX Client and the integrated application are both running on the Login screen. The user logs into the integrated application. To trigger login to IMPAX as well at that point, the integrated application must respond as follows:

Stage	IMPAX Client	Context Server	Integrated application
1			← Set **UserName** property
2		← *UserNameChange* event →	
3			Encrypt the password. ← Set **Password** property
4		← *PasswordChange* event →	
5			← **PerformUserValidation** method
6		← *PerformUserValidationTrigger* event →	
7	Decrypt the password. Log into the application. Set **UserName** property →		
8		← *UserNameChange* event →	
9	Encrypt password. Set **Password** property →		
10		← *PasswordChange* event →	
11	**PerformUserValidation** method →		
12		← *PerformUserValidationTrigger* event →	

State 2: IMPAX Client started by integrated application

The user logs into the integrated application when IMPAX is not running.

The IMPAX Client application can be started by the integrated application. When the integrated application also wants to log the user into the IMPAX Client Application, it has to wait until the IMPAX Client is fully started and connected to the Application Server. Before that point, IMPAX ignores any *PerformUserValidationTrigger* events.

To do this, have the integrated application start the IMPAX Client, set the UserName and (encrypted) Password in the Context Server, then send the *PerformUserValidationTrigger* event. If the event is processed successfully by the IMPAX Client, IMPAX issues its own *PerformUserValidationTrigger* event. The integrated application can wait for this second

event to know whether the login succeeded. If it does not receive the event within a reasonable timeout, it should retry the login.

Stage	IMPAX Client	Context Server	Integrated application
1			Start IMPAX through the operating system (using shell() in VB or Process.Start() in .NET, for example).
2	IMPAX Client starts		
3		← *ScreenAttributeChange* event →	
4		← *ScreenAttributeChange* event →	
5		← *ScreenAttributeChange* event →	
6	Set **UserName** property to ""		
7		← *UserNameChange* event →	
8	Set **Password** property to ""		
9		← *PasswordChange* event →	
10			← Set **UserName** property
11		← *UserNameChange* event →	
12			Encrypt the password. ← Set **Password** property
13		← *PasswordChange* event →	← **PerformUserValidation** method
14		← *PerformUserValidationTrigger* event →	
15	Decrypt the password. Log in. Set **UserName** property →		
16		← *UserNameChange* event →	
17	Encrypt the password. Set **Password** property →		
18		← *PasswordChange* event →	
19	**PerformUserValidation** method →		

Stage	IMPAX Client	Context Server	Integrated application
20		← *PerformUserValidationTrigger* event →	If received, no action. If not received after a period of time, retry.

Setting up single sign-off

With single sign-off, logging out of one integrated application triggers the other applications to log out as well. If you have set up single sign-on, you probably want to set up single sign-off as well.

State 1: User logs out of IMPAX

When a user logs out of the IMPAX Client, the Client clears the UserName and Password property of the Context Server. The integrated application should respond to the associated *PasswordChange* event by checking the values of the UserName and Password. If these are blank, the integrated application should also log out.



Note:

Logging out of the integrated application should be done gracefully, with prompts to save work.

Stage	IMPAX Client	Context Server	Integrated application
1	Clears the **UserName** property.		
2		← *UserNameChange* event →	
3	Clears the **Password** property.		
4		← *PasswordChange* event →	
5			Get the **UserName** and **Password** properties. If both are blank (""), log out gracefully.

State 2: User logs out of the integrated application

When a user logs out of the integrated application, you can set up single sign-off from IMPAX as well by having the integrated application apply the DoAction("LOGOUT", "[<source>]", "", "") method. If any user is logged into the IMPAX Client Application at that time, the DoAction triggers the associated Action() in the Client, and this user is logged out. If no user is logged in, the corresponding event is ignored.



Note:

The DoAction("LOGOUT", "[<source>]", "", "") method does *not* trigger an Action("LOGOUT", "IMPAX", "", "") event.

With the IMPAX Client application, logging out at any time is safe, as long as the IMPAX user profile is set to show a warning message if any unsaved work exists.

Tip:

To enable this warning, log in as or impersonate the user whose profile is to be configured, open the Customize dialog, switch to the User Profile tab, and select the **Prompt to save study** checkbox. For more information, refer to the *IMPAX 6.0 Client Knowledge Base*.

Stage	IMPAX Client	Context Server	Integrated application
1			← DoAction("LOGOUT", "[<source>]", "", "") method
2		← Action("LOGOUT", '', "", "") event →	
3	Logs out user. Clears the **UserName** property.		
4		← **UserNameChange** event →	
5	Clears the **Password** property.		
6		← **PasswordChange** event →	

Selecting Studies for Display

3

This chapter explains how to identify studies and load study images through an integrated application. For details on all events, methods, and properties referenced in this chapter, refer to *Chapter 8: Events Reference on page 48*, *Chapter 9: Methods Reference on page 61*, and *Chapter 10: Properties Reference on page 64*.

Identifying the studies to be displayed

In previous releases of IMPAX, to have an integrated application define which studies are opened in IMPAX, you had to specify the DICOM study UIDs of the studies. This was done by setting study UID through the integrated application database or through a RIS Gateway or PACS broker.

With IMPAX 6.2, you can use VB scripting to identify studies by study UID, patient ID and accession number, or accession number only. The scripts can then be called by the Context Server using the new DoAction RUN_SCRIPT method.

For example, the following script adds a study to the Image area for the specified patient ID and accession number.

```
Option Strict Off

Imports System
Imports System.Windows.Forms

Module Script
      Sub Main()
      ScriptDisplay.AddToDisplayListWithPidAcn("P123", "A456")
      End Sub
End Module
```

Calling study UIDs through a RIS Gateway or PACS broker is no longer recommended, as IMPAX 6.2 Clients do not have direct ODBC connections. However, you can still set the study

UID from the integration application database, if you want. The following code sample illustrates how.

```
Declare global variable StudyIdSet as Boolean


# This is the generic procedure to set the StudyId property using
# patient ID and accession number
Procedure SetStudyIdentifier( Studyuid ) returns Boolean
      # Set the StudyId property to the DICOM Study UID
      ContextServer.StudyId := Studyuid
      # Always return True
      Return( True )
End Procedure
```

The following table shows the order of events when se4tting the study UID from the integrated database application.

Stage	IMPAX Client	Context Server	Integrated application
1			← Set **StudyId** property to the study UID.
2		← **StudyIdentifierChange** event →	

Adding studies to an IMPAX worklist

To build an IMPAX 6.2 worklist, use either the DoAction ADD_TO_DISPLAY_LIST method or the DoAction ADD_TO_DISPLAY_LIST_WITH_PRIORS method to add the identified studies to the Cycle List. You can then use the GETSTUDIESINDISPLAY method to display the studies, as follows.

Stage	IMPAX Client	Context Server	Integrated application
1			← **DoAction("ADD_TO_DISPLAY_LIST"...)** method
2		← **ACTION("ADD_TO_DISPLAY_LIST",...)** event →	
3	Adds the specified studies the cycle list.		
4	**DoAction("STATUS","ADD_TO_DISPLAY_LIST", "<KVPid>","<ResultStatus(es)>")** method →		
5		← **Action("STATUS","ADD_TO_DISPLAY_LIST", "<KVPid>", "<ResultStatus(es)>")** event →	

Cycling studies into the Image area

In past releases of IMPAX, users added studies to a Select worklist before the studies could be opened in Display. With IMPAX 6.2, studies can be *cycled* directly into the Image area without first passing through a List area worklist. *Cycling* means loading studies in the Image area and allowing a user to view them one at a time. By cycling specified studies, you essentially use IMPAX as a "light box" for viewing images, with the integrated application (such as an EPR system) replacing the function performed by the List and Text areas in IMPAX.

Hiding the List and Text areas

To hide the IMPAX List and Text areas, use the DoAction("HIDE_SELECT", "", "", "") method.

Stage	IMPAX Client	Context Server	Integrated application
1			← **DoAction("HIDE_SELECT", "", "", "")** method
2		← ***Action*("HIDE_SELECT", "", "", "")** event →	
3	Hides the List and Text area windows		

Clearing the Image area

Optionally, clear existing studies before loading new studies in the Image area. Do this with the ResetDisplayList method.

Stage	IMPAX Client	Context Server	Integrated application
1			← **ResetDisplayList** method
2		← ***ResetDisplayListTrigger*** event →	
3	Clears the Image area.		

Cycling studies

The studies identified using one of the methods in *Identifying the studies to be displayed* on page 19 can then be loaded in the Image area. To add multiple studies, run this in a loop.

Stage	IMPAX Client	Context Server	Integrated application
1			Perform ← **AddToDisplayList** method
2		← ***AddToDisplayListTrigger*** event →	

Stage	IMPAX Client	Context Server	Integrated application
3	Adds the study to the Image area cycle list. If no study is found with the specified study UID, nothing happens. If the cycle list is empty or the study is for the currently viewed patient, the Image area is displayed. In all other cases, studies are simply added to the cycle list without bringing the Image area into view.		
4	Set **PatientId** property →		
5		← *PatientIdentifierChange* event →	
6	Set **StudyId** property →		
7		← *StudyIdentifierChange* event →	
8	Set **AccessionNumber** property →		
9		← *AccessionNumberChange* event →	
10	Set **StudyStatus** property →		
11		← *StudyStatusChange* event →	

Displaying Study Images

4

This chapter explains how to change the display of studies opened in the Image area. For details on all events, methods, and properties referenced in this chapter, refer to *Chapter 8: Events Reference on page 48*, *Chapter 9: Methods Reference on page 61*, and *Chapter 10: Properties Reference on page 64*.

Clearing the Image area

The integrated application can clear the Image area, so that no images or demographic information are displayed. Unlike previous versions, in IMPAX 6.2 no dialog appears from which a user can then select a different study to display.

To clear the Image area, use the DoAction("BLANK_DISPLAY","","","") method. If the IMPAX Client is not active, the event is ignored.

Stage	IMPAX Client	Context Server	Integrated application
1			← **DoAction("BLANK_DISPLAY", "", "", "")** method
2		← **Action("BLANK_DISPLAY", "", "", "")** event →	
3	**DoAction("STATUS", "BLANK_DISPLAY", "***status***,"")** method →		
4		← **Action("STATUS", "BLANK_DISPLAY","***status***","")** event →	
5	If *status* is 0 (success), the Image area is cleared.		

Another option in IMPAX 6.2 is to use the ResetDisplayList method. This triggers the *ResetDisplayListTrigger* event (page 57), which causes IMPAX 6.2 to clear the Image area.

Stage	IMPAX Client	Context Server	Integrated application
1			← **ResetDisplayList** method
2		← *ResetDisplayListTrigger* event →	
3	The Image area is cleared.		

Note:

The effect of the DoAction - BLANK_DISPLAY and ResetDisplayList methods is identical, except that DoAction - BLANK_DISPLAY raises a STATUS action.

Navigating to a study in the Image area

When studies from different patients are present in the Image area cycle list, only one patient's studies are displayed at a time. A different study can be opened programmatically by applying the DoAction("SYNC_DISPLAY", "<PatientID>", "<AccessionNumber>", "StudyUID") method.

Reference the study to be made visible by its patient ID and accession number, its accession number only, or its Study UID. For details on how to do this, refer to *Identifying the studies to be displayed* on page 19.

IMPAX responds by triggering an Action("STATUS", "SYNC_DISPLAY", "<KVPid>", "<ResultStatus") event in which *ResultStatus* can equal

- 0 for success
- 1 if the study was not in the Cycle List nor in the Set Aside List
- -1 if the study does not exist in IMPAX
- -1 for any other error, for example, error with the parameter specification

If the study cannot be displayed, the Image area does not change. Unlike previous versions, in IMPAX 6.2 a dialog does not appear showing other studies that can be selected.

The integrated application should check that the correct study has been displayed. It can do that by comparing the current value of the StudyId (or AccessionNumber) property to the value for the study that should have been displayed. If failure is indicated, it should retry setting the study reference and perform the DoAction method again, up to a predefined number of times.

When setting up the integration, note the following:

- Setting the StudyUId property clears the AccessionNumber property. The AccessionNumber property is reset by IMPAX only if it effectively displays a new study. So the comparison should be: either the AccessionNumber contains the intended accession number or it is "".
- If IMPAX is not active, this event is ignored.

- If IMPAX is configured to show one study per screen and the requested study is already displayed on the second screen, the displayed study may become visible on both the first and second Image area screens.

A prerequisite is to identify the study to be displayed. In the example below, the study is identified by its DICOM study UID.

Stage	IMPAX Client	Context Server	Integrated application
1			← **DoAction ("SYNC_DISPLAY", "", "", "<StudyUID")** method
2		← **Action("SYNC_DISPLAY", "", "", "<StudyUID>")** event →	
3	**DoAction("STATUS", "SYNC_DISPLAY", "<KVPid>","<ResultStatus")** method →		
4		← **Action("STATUS", "SYNC_DISPLAY", "<KVPid>","<ResultStatus")** event →	
5	If *status* = 0 (success), IMPAX displays that study.		
6			Compare **StudyId** property to the study UID intended to be displayed.

Note:

In previous releases of IMPAX, you could use the DoAction("REMOVE_FROM_WORKLIST", "", "", "") or ("REMOVE_ALL_FROM_WORKLIST", "", "", "") to clear specific or all studies from the Select and Display worklists. In IMPAX 6.2, however, this action works only on the Search and Advanced Search worklists in the List area.

Dictating Studies

5

A common integration is to link IMPAX with reporting software. This chapter explains how an integrated application can respond to IMPAX dictation commands. For details on all events, methods, and properties referenced in this chapter, refer to *Chapter 8: Events Reference on page 48*, *Chapter 9: Methods Reference on page 61*, and *Chapter 10: Properties Reference on page 64*.

Launching a dictation application from IMPAX

Commonly users with dictation privileges should be authenticated in both IMPAX and the integrated reporting software at the same time. For details on how to set up this "single sign-on", refer to *Chapter 2: Logging into IMPAX* on page 12.

Understanding dictation status

Each IMPAX study has a dictation status. When a study arrives in the IMPAX Archive, its status is *N* for *New*. As reporting actions are completed in the system, its status changes. When the status is *d* or *t* (dictation in progress) or *D* or *T* (dictated), other users in the system are prevented from reporting on the study.

IMPAX reporting statuses

Status	Code	Button appearance
New	N	
Dictation in progress	d	
Trainee dictation in progress	t	
Dictated	D	
Trainee dictated	T	

IMPAX reporting statuses

Status	Code	Button appearance
Reported	r	
Approved	R	

The Dictate button in the Image area can be set to two-state or three-state mode.

- In two-state mode, a study goes directly from *New* to *Dictated* or *Trainee dictated* when a user clicks the Dictate button. This mode is most often used with non-integrated systems, in which requisition forms are used to track in-progress dictations.
- In three-state mode, a study changes from *New* to *Dictation in Progress* or *Trainee dictation in progress* status on the first click, then to *Dictated* or *Trainee dictated* on the second click. Three-state mode is most commonly used for digital dictation.

Note:

IMPAX sets whether or not the *Trainee* status is used based on the user's access to a study. For example, a user may dictate as a trainee for studies from some modalities but not others.

If the appropriate preference is selected in the Configure area - Preferences section, once a study is dictated, IMPAX automatically displays the next undictated study.

Dictating from the IMPAX Client

Most commonly, radiologists start dictation in IMPAX, and the integrated reporting application is synchronized. Slight variations occur based on whether a single study or multiple patient studies are being dictated at once, and whether two-state or three-station dictation mode is used. For more information on these modes, refer to *Understanding dictation status* on page 26.

Note:

IMPAX may also raise the *PatientIdentifierChange, StudyIdentifierChange, AccessionNumberChange,* and *StudyStatusChange* events before the DICTATION_STARTED and DICTATION_COMPLETED actions—if these properties are changed by that action.

Starting single-study dictation in three-state mode

A study is displayed in the IMPAX Image area. A user with dictation privileges clicks the Dictate button. Assuming three-state dictation is being used, the integrated reporting application should respond as follows:

Stage	IMPAX Client	Context Server	Integrated application
1		← *Action*("DICTATION_STARTED", "*study_UID*", "*accession_number*", "") event →	
2			Start the dictation interface. Get the **PatientId** and **AccessionNumber** (or **StudyId**) properties of the study to dictate.
3	Set **StudyStatus** property to *d* or *t*. →		

Ending single-study dictation in three-state mode

When the report is completed, the user clicks the Dictate button in the Image area again. The integrated reporting application should respond as follows:

Stage	IMPAX Client	Context Server	Integrated application
1	DoAction("DICTATION_COMPLETED", "*study_UID*", "", "") method →		
2		← *Action*("DICTATION_COMPLETED", "*study_UID*", "*accession_number*", "") event →	
3	Set **StudyStatus** property to *D* or *T*. →		

Starting and ending single-study dictation in two-state mode

Though not typically used for integrated dictation systems, this is how messages are passed when two-state mode is configured and the user clicks the Dictate button in IMPAX.

Stage	IMPAX Client	Context Server	Integrated application
1	DoAction("DICTATION_STARTED", "*study_UID*", "", "") method →		

Stage	IMPAX Client	Context Server	Integrated application
2		← ***Action*** **("DICTATION_STARTED",** *"study_UID"*, *"accession_number"*, **"")** event →	
3	Set **StudyStatus** property to *d* or *t*. →		
4		← ***StudyStatusChange*** event →	
5	**DoAction("DICTATION_COMPLETED",** *"study_UID"*, **"", "")** method →		
6		← ***Action*** **("DICTATION_COMPLETED",** *"study_UID"*, *"accession_number"*, **"")** event →	
7			Launch dictation application. Get the **PatientId** and **AccessionNumber** (or **StudyId**) properties of the study to be dictated. User must indicate the end of dictation inside the integrated application.
8	Set **StudyStatus** property to *D* or *T*. →		
9		← ***StudyStatusChange*** event →	

Starting multiple-study dictation in three-state mode

A user can also select multiple studies from a patient to dictate, then click the Dictate button in the Image area. The resulting report applies to all selected studies. The communication in this scenario is very similar to single-study dictation in three-state mode.

Stage	IMPAX Client	Context Server	Integrated application
1		← ***Action*****("DICTATION_STARTED",** *"study_UIDs"*, *"accession_numbers"*, **"")** method (multiple values, each separated by a backslash) →	

Stage	IMPAX Client	Context Server	Integrated application
2			Start the dictation interface. Get the **PatientId** and **AccessionNumber** (or **StudyId**) properties of the studies to be dictated.
3	Set **StudyStatus** property (multiple values separated by backslashes) to *d* or *t*. →		
4		← *StudyStatusChange* event →	

When the user clicks the Dictate button again to indicate the end of dictation, the resulting communication is the same as for single-study dictation (page 28), except that multiple study UIDs and accession numbers, each separated by a backslash, are communicated.

Dictating from an integrated application

You can also support dictation from the integrated reporting application by sending the study status information back to IMPAX. When the user selects a study and starts dictation in the integrated application, you may want to have IMPAX display the study in the Image area. Set that up as described in *Chapter 3: Selecting Studies for Display* on page 19.

The integrated reporting application should respond as follows (assuming three-state dictation mode):

Stage	IMPAX Client	Context Server	Integrated application
1			← **DoAction("START_DICTATION", "***study_UID***", "", "")** method
2		← ***Action*****("DICTATION_STARTED", "***DICOM study UID***", "***accession number***", "")** event →	
3			Get the **PatientId** and **AccessionNumber** (or **StudyId**) properties (or get it from the **Action** event parameters) to know which study will be dictated.
4	Set **StudyStatus** property to *d* or *t*. →		
5		← *StudyStatusChange* event →	

Ending dictation from an integrated application

When the user indicates that the report is finished in the integrated application, the following messages can be launched (in three-state dictation mode).

Stage	IMPAX Client	Context Server	Integrated application
1			← **DoAction("COMPLETE_DICTATION","*study_UID*","","")** method
2		← ***Action*** **("COMPLETE_DICTATION","","","")** event →	
3		← ***Action*****("DICTATION_COMPLETED","*DICOM study UID*","*accession number*","")** event →	
4	Set **StudyStatus** property to *D* or *T*. →		
5		← ***StudyStatusChange*** event →	

Canceling dictation from an integrated application

A user may also choose to cancel dictation rather than complete the report in the integrated application. Use the UPDATE_STUDY_STATUS attribute of the DoAction action to change the status of the study from *Dictation in Progress* back to *New*.

Stage	IMPAX Client	Context Server	Integrated application
1			← **DoAction("UPDATE_STUDY_STATUS","*study_UID*","*accession_number*","N")** method
2		← ***Action*****("UPDATE_STUDY_STATUS","*study_UID*","*accession_number*","N")** event →	
9	Set **StudyStatus** property to *N*. →		
10		← ***StudyStatusChange*** event →	
11	**DoAction("STATUS","UPDATE_STUDY_STATUS","*status*","")** method →		
12		← **Action("STATUS","UPDATE_STUDY_STATUS","*status*","")** event	

Stage	IMPAX Client	Context Server	Integrated application
13			Possibly use this event to check the *status*.

Creating Scripts

6

IMPAX 6.2 introduces support for VB.NET scripts that can be invoked through a toolbar in the List area or a Context Server DoAction command. The script interface is fully described in *Appendix A: IMPAX 6.2 Script Interfaces* on page 73. This chapter provides more information about VB.NET scripting.

Scripting basics

Scripting in the IMPAX 6.2 Client uses VB.NET as the scripting language. The scripts are executed using the Visual Studio for Applications (VSA) runtime engine, which is part of the Microsoft .NET Framework.

In the background, the script code is compiled in memory and executed. At runtime, the script is executed as a full VB.NET application, with access to the IMPAX Client application through a set of custom objects.

Once created, you can invoke a script through the Wizard toolbar (for details, refer to the IMPAX 6.0 Client Knowledge Base) or a Context Server DoAction command such as:

```
DoAction("RUN_SCRIPT", "MyScript", "c:\temp\Script.vb", "")
```

Writing scripts

With an Administrator license, you can enter scripts directly in the List area of the IMPAX Client application. Alternatively, you can create scripts in any text editor or in a dedicated application, such as Visual Studio.NET.

Each component in the application that can be scripted implements a script interface. For more information on the main controller classes for each component, refer to *Appendix A: IMPAX 6.2 Script Interfaces* on page 73.

Script code must be valid VB.NET code and must be in a module named “Script”. Give each script a unique name. When the code is executed, the IMPAX Client calls the script’s Main() method.

Script examples

This script clears the Image area and activates the List area.

```
Option Strict Off

Imports System
Imports System.Windows.Forms

Module Script
      Sub Main()
      ScriptDisplay.ClearDisplay()
      End Sub
End Module
```

This script displays the contents of the Search or Advanced Search worklist in the Image area.

```
Option Strict Off

Imports System
Imports System.Windows.Forms

Module Script
      Sub Main()
      ScriptListArea.DisplayWorkList()
      End Sub
End Module
```

This script adds a study to the Image area for the indicated patient ID and accession number.

```
Option Strict Off

Imports System
Imports System.Windows.Forms

Module Script
      Sub Main()
      ScriptDisplay.AddToDisplayListWithPidAcn("P123", "A456")
```

```
      End Sub
End Module
```

This script gets values from a worklist and builds a URL.

```
Option Strict Off

Imports System
Imports System.Windows.Forms

Module Script
      Sub Main()
      dim patientId as string
      dim accessionNumber as string
      dim url as string

      ' get the patient ID and accession number from the currently
      ' selected row in the Search or Advanced Search worklist

      patientId = ScriptListArea.GetValueFromWorkList("patient",
      "patient_id")
      accessionNumber = ScriptListArea.GetValueFromWorkList("study",
      "accession_number")

      ' build a url

      url = "http://ris/ShowOrder.aspx?patientId=" + patientId +
      "&accessionNumber=" + accessionNumber

      ' show the url

      System.Diagnostics.Process.Start(url)
      End Sub
End Module
```

DoAction() Methods and Action() Events

7

This chapter describes the Context Server DoAction() methods and the corresponding Action() events. Events are messages sent by the Context Server in response to Method or Property changes in the IMPAX Client or the integrated application.

ADD_TO_DISPLAY_LIST

ADD_TO_DISPLAY_LIST adds studies to the Cycle list without displaying them.

Method syntax

The third-party integrated application calls the ADD_TO_DISPLAY_LIST DoAction() method with one of these three variations.

Syntax options	Description
DoAction("ADD_TO_DISPLAY_LIST", "<PatientID(s)>","AccessionNumber(s)","")	Add one or more studies, identified by Patient ID and Accession Number, to the Cycle List. Multiple values are separated by a backslash ("\"). Both parameters should hold an equal number of subvalues. Subvalues at the same relative location within each parameter should correspond to the same study.
DoAction("ADD_TO_DISPLAY_LIST","", "AccessionNumber(s)","")	Add one or more studies, identified by accession Number, to the Cycle List. Multiple values are separated by a backslash ("\").
DoAction("ADD_TO_DISPLAY_LIST","","", "StudyUid(s)")	Add one or more studies, identified by accession Number, to the Cycle List. Multiple values are separated by a backslash ("\").

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Silently adds the referenced studies to the Cycle list. “Silently” means that the current Image area display remains unchanged.
- Triggers the following event:

```
Action("STATUS","ADD_TO_DISPLAY_LIST","KVPid","ResultStatus")
```

Condition	KVPid	ResultStatus
All referenced studies successfully added	KVPid of referenced studies	0\...\0
Some referenced studies added successfully; others were not (because the studies do not exist in IMPAX)	KVPid of referenced studies, with unsuccessful studies having an empty KVPid	0 = successful 1 = failed
Other errors, such the number of patient IDs not matching the number of accession number	Empty KVPid	-1

ADD_TO_DISPLAY_LIST_WITH_PRIORS

ADD_TO_DISPLAY_LIST adds studies and their priors to the Cycle list without displaying them.

Method syntax

The third-party integrated application calls the ADD_TO_DISPLAY_LIST_WITH_PRIORS DoAction() method with one of these three variations

Syntax options	Description
DoAction(”ADD_TO_DISPLAY_LIST_WITH_PRIORS”, ”<PatientID(s)>”, ”AccessionNumber(s)”,””)	Add one or more studies, identified by Patient ID and Accession Number, to the Cycle list. Multiple values are separated by a backslash (“\”). Both parameters should hold an equal number of subvalues. Subvalues at the same relative location within each parameter should correspond to the same study.
DoAction(”ADD_TO_DISPLAY_LIST_WITH_PRIORS”,””, ”AccessionNumber(s)”,””)	Add one or more studies, identified by accession Number, to the Cycle list. Multiple values are separated by a backslash (“\”).
DoAction(”ADD_TO_DISPLAY_LIST_WITH_PRIORS”,””,””, ”StudyUid(s)”)	Add one or more studies, identified by accession Number, to the Cycle list. Multiple values are separated by a backslash (“\”).

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Silently adds the referenced studies to the Cycle list. “Silently” means that the current Image area display remains unchanged.
- Triggers the following event:

```
Action("STATUS", "ADD_TO_DISPLAY_LIST_WITH_PRIORS", "KVPid"
"ResultStatus")
```

Condition	KVPid	ResultStatus
All referenced studies successfully added	KVPid of referenced studies	0\...\0
Some referenced studies added successfully; others were not (because the studies do not exist in IMPAX)	KVPid of referenced studies, with unsuccessful studies having an empty KVPid	0 = successful 1 = failed
Other errors, such the number of patient IDs not matching the number of accession number	Empty KVPid	-1

BLANK_DISPLAY

BLANK_DISPLAY clears the Image area.

Method syntax

The third-party integrated application calls the BLANK_DISPLAY DoAction() method as follows:

```
DoAction("BLANK_DISPLAY","","","")
```

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Clears the Image area, including all demographic information. Studies in the Cycle list and the set aside list remain as is.
- Triggers the following event:

Action("STATUS","BLANK_DISPLAY","","*ResultStatus*")

ResultStatus values:

- 0 = success
- 1 = failure

For more information, refer to *Clearing the Image area on page 23*.

DICTATION_STARTED

DICTATION_STARTED begins a dictation for the specified study or studies.

Method syntax

The third-party integrated application calls the DICTATION_STARTED DoAction() method as follows:

```
DoAction("DICTATION_STARTED","StudyUID(s)","AccessionNumber(s)",
"")
```

Separate multiple Study UIDs and Accession Numbers, if specified, by a backslash.

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Starts a dictations for the specified study or studies.The only requirement is that the logged-in user have permission to dictate. IMPAX 6.2 does not check the status of the study when receiving this action, and the study does not have to be displayed.
- IMPAX triggers the *PatientIdentifierChange*, *StudyIdentifierChange*, *AccessionNumberChange*, and *StudyStatusChange* events before the DICTATION_STARTED action—if that information is changed by this action.

For more information, refer to *Dictating from the IMPAX Client on page 27* and *Dictating from an integrated application* on page 30.

DICTATION_COMPLETED

DICTATION_STARTED begins a dictation for the specified study or studies.

Method syntax

The third-party integrated application calls the DICTATION_STARTED DoAction() method as follows:

```
DoAction("DICTATION_COMPLETED","StudyUID(s)","AccessionNumber(s)",
"")
```

Separate multiple Study UIDs and Accession Numbers, if specified, by a backslash.

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Starts a dictations for the specified study or studies.The only requirement is that the logged-in user have permission to dictate. IMPAX 6.2 does not check the status of the study when receiving this action, and the study does not have to be displayed.
- IMPAX triggers the *PatientIdentifierChange*, *StudyIdentifierChange*, *AccessionNumberChange*, and *StudyStatusChange* events before the DICTATION_COMPLETED action—if that information is changed by this action.

For more information about completing dictation, refer to *Ending single-study dictation in three-state mode on page 28* and to *Ending dictation from an integrated application* on page 31.

GETSTUDIESINDISPLAY

GETSTUDIESINDISPLAY specifies studies that are currently visible in the Image area, loaded in the Cycle list, or loaded in the set aside list.

Method syntax

The third-party integrated application calls the GETSTUDIESINDISPLAY DoAction() method as follows:

```
DoAction("GETSTUDIESINDISPLAY","","","")
```

IMPAX Client response

When the IMPAX Client receives the associated Action() event, it triggers this event:

```
ACTION("GETSTUDIESINDISPLAYRESULT","","<KVPid1>","<KVPid2>")
```

Parameter	Description
KVPid1	*KVPid1* specifies all studies that are visible in the Image area, whether active or not.
KVPid2	*KVPid2* specifies all studies that are either set aside or included in the Cycle list.

GETUIDBYPICACN and GETUIDSBYPID

GETUIDBYPICACN and GETUIDSBYPID have IMPAX return the DICOM Study UID for the specified study.

Method syntax

The third-party integrated application calls the DoAction() method with one of these variations:

Syntax	Value
DoAction(”GETUIDBYPICACN”, ”<PatientID>”,”<AccessionNumber>”,”<StudyUID>”)	Have IMPAX return the DICOM Study UID for the study specified by the Patient ID and Accession Number. Multiple Accession Numbers and Study UIDs are separated by a backslash (\).
DoAction(”GETUIDSBYPID”.”<Patient ID>”,””,””)	Have IMPAX return a list of the DICOM Study UIDs for the study specified by the patient ID.

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- `Action("GETUIDBYPINDACN", ...)` returns the DICOM study UID for the study specified by the Patient ID and Accession Number.
- `Action("GETUIDBYPID", ...)` returns a list of the DICOM study UIDs for the study specified by the Patient ID.

HIDE_SELECT and SHOW_SELECT

HIDE_SELECT and SHOW_SELECT hide or show the List and Text areas.

Method syntax

The third-party integrated application calls the DoAction() method with one of these variations:

Syntax options	Description
DoAction("HIDE_SELECT", "", "", "")	Hide windows and remove them from the Windows Taskbar. They remain running in the background.
DoAction("SHOW_SELECT", "", "", "")	Show windows and have them appear in the Windows taskbar.

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- `Action("HIDE_SELECT", ...)` hides windows and removes them from the Windows Taskbar.
- `Action("SHOW_SELECT", ...)` shows windows and has them appear in the Windows Taskbar.

For an example, refer to *Hiding the List and Text areas on page 21*.

LOGOUT from Integrated Application

LOGOUT, in this situation, logs out from the Integrated Application. It can also log out of IMPAX: refer to *LOGOUT from IMPAX Client application on page 41*.

Method syntax

The third-party integrated application calls the LOGOUT DoAction() method with the following syntax:

```
DoAction("LOUGOUT","[<Source>]","","")
```

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client logs out. It does *not* trigger an event that logs out of IMPAX. Refer to *LOGOUT from IMPAX Client application on page 41*.

LOGOUT from IMPAX Client application

LOGOUT, in this situation, logs out of the IMPAX Client application.

User action

When a user manually logs out of the IMPAX Client Application or when the IMPAX Client Application is automatically logged out after a time-out.

IMPAX Client response

The IMPAX Client triggers the following event and logs out:

```
Action("LOGOUT","IMPAX","","")
```

REMOVE_ALL_FROM_CYCLE_LIST

REMOVE_ALL_FROM_CYCLE_LIST removes studies from the Cycle list.

Method syntax

The third-party integrated application calls the REMOVE_ALL_FROM_CYCLELIST DoAction() method with the following syntax:

```
DoAction("REMOVE_ALL_FROM_CYCLE_LIST","","","")
```

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Removes all studies from the Cycle List. The Set Aside List and the Advanced Search List in the List Area are unaffected. If there are no studies in the Set Aside List, the Image Area is de-activated; otherwise, one or more of the studies in the Set Aside List is activated.
- Triggers the following event:

```
Action("STUDY_CHANGED","contextserver","<empty KVPid>","")
```

REMOVE_ALL_FROM_WORKLIST

REMOVE_ALL_FROM_WORKLIST removes studies from the Cycle list.

Method syntax

The third-party integrated application calls the REMOVE_ALL_FROM_WORKLIST DoAction() method with the following syntax:

```
DoAction("REMOVE_ALL_FROM_WORKLIST","","","")
```

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Removes all studies from the Cycle List and the Set Aside List. As a result, the Image area is de-activated. The Advanced Search List in the List Area is unaffected.
- Triggers the following event indirectly:

```
Action("STUDY_CHANGED","contextserver","<empty KVPid>","")
```

ROLLBACK_IN_PROGRESS_STUDY

ROLLBACK_IN_PROGRESS_STUDY changes the status for the specified *Dictation in Progress* or *Trainee Dictation in Progress* to *New*.

Method syntax

The third-party integrated application calls the ROLLBACK_IN_PROGRESS_STUDY DoAction() method as follows:

```
DoAction("ROLLBACK_IN_PROGRESS_STUDY","<StudyUID(s)">,"","")
```

Separate multiple Study UIDs with a backslash (\).

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs this task:

- Updates the status for the specified dictation to *New*.

RUN_SCRIPT

RUN_SCRIPT executes the specified VB.NET script.

Method syntax

The third-party integrated application calls the RUN_SCRIPT DoAction() method with one of these three variations:

Syntax options	Description
Action("RUN_SCRIPT", "<ScriptName>", "", "")	Executes the code associated with the Script Name.
Action("RUN_SCRIPT", "<ScriptName>", "<ScriptFileName>", "")	Attempts to use the value of ScriptFileName as a file name: it loads the file and executes the code inside it. The file name may be a complete path; for example, C:\Scripts\ShowStudyFromRis.vb.
Action("RUN_SCRIPT", "<ScriptName>", "<ScriptFileName>", "<ScriptCode>")	Attempts to use the contents of the argument as the code to run.

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Attempts to run the specified script

For more information, refer to *Chapter 6: Creating Scripts* on page 33 and *Appendix A: IMPAX 6.2 Script Interfaces* on page 73.

SET_FOCUS

SET_FOCUS causes IMPAX to take focus and become the foreground application.

Method syntax

The third-party integrated application calls the SET_FOCUS DoAction() method as follows:

```
DoAction("SET_FOCUS","IMPAX","","")
```

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs this task:

- takes focus and becomes the foreground application.

SET_WINDOW_STATE

SET_WINDOW_STATE causes IMPAX to take focus and become the foreground application.

Method syntax

The third-party integrated application calls the SET_WINDOW_STATE DoAction() method with one of these variations:

Syntax options	Description
Action(″SET_WINDOW_STATE″, ″0″, ″″, ″″)	The List and Text area window change state to normal and sizeable; the Image area window does not change.
Action(″SET_WINDOW_STATE″, ″1″, ″″, ″″)	The List and Text area window change state to minimized; the Image area window becomes invisible.
Action(″SET_WINDOW_STATE″, ″2″, ″″, ″″)	The List and Text area window change state to maximized; the Image area window does not change.

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs this task:

- Changes the window state of the List and Text area.

STUDY_CHANGED

STUDY_CHANGED is an Action with no directly corresponding DoAction method.

IMPAX Client response

STUDY_CHANGED can be triggered as a result of a user action in the IMPAX Client application or as a result of a Context Server method.

Syntax options	Description
ACTION("STUDY_CHANGED", "IMPAX", "*KVPid*", ""	Triggered when a user activates a study in the IMPAX Client application, either by selecting a set aside study or a study in the Cycle list, or by displaying List area studies when the Image area is not yet active. The *KVPid* contains all new active studies. If the Image area is no longer displaying studies, the *KVPid* is empty.
ACTION("STUDY_CHANGED", "CONTEXTSERVER", "*KVPid*", ""	Triggered in response to a DoAction method that activates a study, such as DoAction(SYNC_DISPLAY,...). The *KVPid* contains all new active studies. If the Image area is no longer displaying studies, the *KVPid* is empty, as is the case with DoAction("REMOVE_ALL_FROM_WORKLIST", ...).



Note:

When the Image area is not active and the integrated application adds a study to the Cycle List through DoAction("ADD_TO_DISPLAY_LIST",...), the study is added "silently" and is not actually displayed. No Action("STUDY_CHANGED",...) event is triggered in this case.

SYNC_DISPLAY

SYNC_DISPLAY attempts to bring the specified study forward to the left-hand moniter.

Method syntax

The third-party integrated application calls the SYNC_DISPLAY DoAction() method with one of these four variations

Syntax options	Description
DoAction("SYNC_DISPLAY", "", "","")	Displays the study referenced by the DICOM Study UID value specified in the StudyId property of the Context Server. If the StudyId property has multiple values, the first values is used.
DoAction("SYNC_DISPLAY", "<PatientID>", "AccessionNumber","")	Displays the study specified by <PatientID> and <AccessionNumber>. Only a single patient ID and Accession Number may be identified.
DoAction("SYNC_DISPLAY","", "AccessionNumber(s)","")	Displays the study specified by <AccessionNumber>. Only a single Accession Number may be identified.
DoAction("SYNC_DISPLAY","","", "StudyUid(s)")	Displays the study referenced by <StudyUid>. Only a single Study UID may be identified.

IMPAX Client response

When the IMPAX Client receives the associated Action("SYNC_DISPLAY",…) event, the IMPAX Client performs these tasks

- Activates the Image area, if it is not yet active.
- Attempts to bring forward (make visible) on the left-hand monitor the specified study. Any other studies or monitors are displayed as if the user had manually positioned the specified study. The Text area and List area remain unchanged.
- Makes the referenced study active, and triggers the following event:

```
Action("STATUS", "SYNC_DISPLAY", "KVPid", "ResultStatus")
```

Condition	KVPid	ResultStatus
Referenced study can be displayed	KVPid of referenced study	0
Referenced study is not set aside and is not in Cycle list	KVPid of referenced study	1
Referenced study does not exist in IMPAX	KVPid of referenced study, with only the values specified in the originating DoAction method	-1
Any other error, such as a syntax error	Empty KVPid	-1

  If the study is not already set aside or in the Cycle list, the Image area does not change. The integrating application can optionally blank the Image area on receiving an error status event.

- If the referenced study was not previously active, it also triggers this event indirectly:

```
Action("STUDY_CHANGED", "contextserver","<KVPid>","")
```

UPDATE_STUDY_STATUS

ADD_TO_DISPLAY_LIST adds studies to the Cycle list without displaying them.

Method syntax

The third-party integrated application calls the UPDATE_STUDY_STATUS DoAction() method as follows:

```
DoAction("UPDATE_STUDY_STATUS","<StudyUID>","<AccessionNumber>",
"<StudyStatus>")
```

IMPAX Client response

When the IMPAX Client receives the associated Action() event, the IMPAX Client performs these tasks:

- Updates the status for the specified study.
- Triggers the following event:

```
Action("STATUS","UPDATE_STUDY_STATUS","","ResultStatus")
```

Condition	ResultStatus
Success	0
Study not found in worklist (applicable only to IMPAX 4.5 and 5.*x*)	1
Multiple studies found for given study UID or accession number	2
Unspecified error	-1

We recommend that you use this event to verify that the UPDATE_STUDY_STATUS call succeeded. For an example, refer to *Canceling dictation from an integrated application on page 31*.

Events Reference

8

This chapter describes the Context Server events. Events are messages sent by the Context Server in response to Method or Property changes in IMPAX or the integrated application.

AccessionNumberChange

Occurs when the Context Server receives a change to the following property:

- *AccessionNumber, page 64*

Syntax

Event AccessionNumberChange (ByVal *accession_number* **As String)**

Part	Data type	Description
accession_number	String	An identifier that RIS and PACS systems agree on as an accession number.

IMPAX behavior

IMPAX does nothing on receiving this event.

Action

Occurs when the Context Server receives an execution of the DoAction method. Refer to Chapter 7: *SYNC_DISPLAY*.

AddToDisplayListTrigger

Occurs when the Context Server receives an execution of the following method:

- *AddToDisplayList, page 61*

Syntax

Event AddToDisplayListTrigger()

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select appends the study referenced by the StudyID property to the Display list.

IMPAX 6.2 adds the study referenced by the StudyID property to the Cycle list.

AlwaysOnTopChange

Occurs when the Context Server receives a change to the following property:

- *AlwaysOnTopFlag, page 64*

Syntax

Event AlwaysOnTopChange (ByVal -1 | 0 As Boolean)

Value	Data type	Description
-1	Long	True: always make the Client application the topmost window.
0	Long	False: do not make the Client application always appear as the topmost window

IMPAX Behavior

IMPAX does nothing on receiving this event.

ApplicationNameChange

Occurs when the Context Server receives a change to the following property:

- *ApplicationNameChange, page 49*

Syntax

Event ApplicationNameChange(ByVal *exe_name* **As String)**

Part	Data type	Description
exe_name	String	If the Client applications are standalone Windows-based EXEs, this property identifies the path and file name of the program to launch.

IMPAX behavior

IMPAX does nothing on receiving this event.

FontChange

Occurs when the Context Server receives changes to any of following properties:

- *FontName, page 66*
- *FontSize, page 67*

Syntax

Event FontChange(*font_name***, *font_size*)**

Part	Data type	Description
font_name	String	The name of the font (for example, Arial, Courier New, or MS Sans Serif).
font_size	Long	The point size of the font (for example, 8, 12, or 20).

IMPAX behavior

IMPAX does nothing on receiving this event.

IconSizeChange

Occurs when the Context Server receives a change to the following property:

- *IconSize, page 67*

Syntax

Event IconSizeChange(ByVal *icon_size* As String)

Part	Data type	Description
icon_size	String	The size of the icon that should be shown.



Note:

IMPAX 4.5 and 5.*x* sends icon sizes as SMALL, MEDIUM, or LARGE, using a 40x40 pixel 16-color bitmap for small, a 60x60 pixel 16-color bitmap for medium, and a 80x80 pixel 16-color bitmap for large. We recommend that other applications follow this convention as well.

IMPAX behavior

IMPAX does nothing on receiving this event.

LaunchApplicationTrigger

Occurs when the Context Server receives an execution of the following method:

- *ALaunchApplication, page 62*

Syntax

Event LaunchApplicationTrigger()

IMPAX Behavior

IMPAX does nothing on receiving this event.



Tip:

Use the value stored in the property ApplicationName in the Context Server to write code in this event to launch the correct application.

PasswordChange

Occurs when the Context Server receives a change to the following property:

- *Password, page 68*

Syntax

Event PasswordChange (ByVal *password_string* **As String)**

Part	Data type	Description
password_string	String	The password to be used for Client applications that support user authentication.

IMPAX behavior

IMPAX does nothing on receiving this event.

PatientIdentifierChange

Occurs when the Context Server receives a change to the following property:

- *PatientId, page 69*

Syntax

Event PatientIdentifierChange (ByVal *patient_id* **As String)**

Part	Data type	Description
patient_id	String	An identifier that RIS and PACS systems agree on as a patient identifier.

IMPAX behavior

IMPAX does nothing on receiving this event.

PerformUserValidationTrigger

Occurs when the Context Server receives an execution of the following method:

- *PerformUserValidation, page 62*

Syntax

Event PerformUserValidationTrigger()

IMPAX behavior

When IMPAX receives this event, it logs in with the *UserName* and *Password* properties to perform user validation on IMPAX. When the Client is running under Korean, Japanese, Vietnamese, Thai, or Chinese versions of Windows, IMPAX expects the password contents to be encrypted using a Unicode locale. In other cases, it uses an ANSI method. It will encrypt the password using one of these methods before setting it into the Context Server.



Tip:

Use the values stored in the properties UserName and Password in the Context Server to write code in this event to perform user authentication on the Client Application Server.

The following is a Microsoft C# code sample for how to use the mitra_crypto.dll with IMPAX 6.2 in Unicode locales:

```
/// <summary>
/// function declaration for the encrypt method in the
mitra_crypto.dll
/// </summary>
/// <param name="password">the password to encrypt</param>
/// <param name="encryptedPassword">the encrypted version of the
/// password</param>
/// <param name="length">the byte length of the password to
/// encrypt</param>
/// <returns>zero if the function fails, non-zero if it
/// succeeds</returns>
[DllImport("mitra_crypto.dll", CharSet=CharSet.Unicode)]
private static extern int EncryptUnicodeOk(string password,
StringBuilder encryptedPassword, int length);

/// <summary>
/// encrypt the password
/// </summary>
```

```
/// <param name="rawPassword">the raw password</param>
/// <returns>the encrypted password</returns>
private string EncryptPassword(string rawPassword)
{
// make sure we have a good value to start with

            if ((rawPassword == null) || (rawPassword.Length ==
0))
            return "";

            // create a buffer to hold the encrypted password

            StringBuilder encryptedPassword = new
StringBuilder(64);

            // encrypt it - we need to pass the byte length of
the
            // password, rather than the number of characters,
so
            // multiply the length by two (two bytes per
Unicode
            // character)

            int ok = EncryptUnicodeOk(rawPassword,
encryptedPassword,
rawPassword.Length * 2);

            // if it failed, return an empty string, otherwise
return
            // the encrypted password

            if (ok == 0)
                  return "";
            else
                  return encryptedPassword.ToString();
}

/// <summary>
/// function declaration for the decrypt method in the
mitra_crypto.dll
/// </summary>
```

```
/// <param name="encryptedPassword">the encrypted password</param>
/// <param name="decryptedPassword">the decrypted version of the
/// password</param>
/// <param name="length">the length of the buffer to hold the
decrypted
/// password (must be MOD 8)</param>
/// <returns>zero if the function fails, non-zero if it
/// succeeds</returns>
[DllImport("mitra_crypto.dll", CharSet=CharSet.Unicode)]
private static extern int DecryptUnicodeOk(string
encryptedPassword,
                StringBuilder decryptedPassword, int length);


/// <summary>
/// decrypt the password
/// </summary>
/// <param name="encryptedPassword">the encrypted password</param>
/// <returns>the decrypted password</returns>
private string DecryptPassword(string encryptedPassword)
{
                // make sure we have a good value to start with

                if ((encryptedPassword == null) ||
(encryptedPassword.Length == 0))
                {
                      return "";
}

                // create a buffer to hold the encrypted password

                StringBuilder decryptedPassword = new
StringBuilder(64);

                // decrypt it

                int ok = DecryptUnicodeOk(encryptedPassword,
decryptedPassword, 64);

                // if it failed, return an empty string, otherwise
return
```

```
                // the decrypted password

                if (ok == 0)
                      return "";
                else
                      return decryptedPassword.ToString();
}
```

The following is a Microsoft C# code sample for how to use the mitra_crypto.dll with IMPAX 6.2 in ANSI locales.

```
/// <summary>
/// encrypt the password
/// </summary>
/// <param name="password">the raw password</param>
/// <param name="encryptedPassword">the encrypted password</param>
/// <param name="length">the length of the password</param>
/// <returns>the length of the encrypted password</returns>
[DllImport("mitra_crypto.dll", CharSet=CharSet.Ansi)]
private static extern int Encrypt(string password, StringBuilder
encryptedPassword,
      int length);

/// <summary>
/// encrypt the password
/// </summary>
/// <param name="rawPassword">the raw password</param>
/// <returns>the encrypted password</returns>
private string EncryptPasswordAnsi(string rawPassword)
{
      if ((rawPassword == null) || (rawPassword.Length == 0))
      return "";

      // create a buffer to hold the encrypted password

      StringBuilder encryptedPassword = new StringBuilder(new
      string(' ', 64));

      // encrypt the password. The return length is the length of
      // the password that has been encrypted. It will be padded
```

```
            // with nulls. If it's zero, the encryption failed

            int returnLength = Encrypt(rawPassword, encryptedPassword,
            rawPassword.Length);

            if (returnLength != 0)
            {
            // pad this to the return length, if it's shorter

            if (encryptedPassword.Length < returnLength)
        {
            string pad =
            encryptedPassword.ToString().PadRight(returnLength, ' ');

            return pad.Substring(0, returnLength);
            }
            else
            {
            return encryptedPassword.ToString().Substring(0,
            returnLength);
            }
            }

            return "";
            }

        /// <summary>
        /// decrypt the password
        /// </summary>
        /// <param name="encryptedPassword">the encrypted password</param>
        /// <param name="decryptedPassword">the decrypted password</param>
        /// <param name="length">the length of the password</param>
        /// <returns>the length of the decrypted password</returns>
        [DllImport("mitra_crypto.dll", CharSet=CharSet.Ansi)]
        private static extern int Decrypt(string encryptedPassword,
            StringBuilder decryptedPassword, int length);

        /// <summary>
        /// decrypt the password
        /// </summary>
```

```
/// <param name="encryptedPassword">the encrypted password</param>
/// <returns>the decrypted password</returns>
private string DecryptPasswordAnsi(string encryptedPassword)
{
      if ((encryptedPassword == null) || (encryptedPassword.Length
         == 0))
         return "";

         // create a buffer to hold the decrypted password

         StringBuilder decryptedPassword = new StringBuilder(new
            string(' ',
         encryptedPassword.Length));

// decrypt the password. The return length is the length of the
// password that has been decrypted. It will be padded with nulls.
// If it's zero, the decryption failed

int returnLength = Decrypt(encryptedPassword, decryptedPassword,
encryptedPassword.Length);

               if (returnLength != 0)
               {
                     return
decryptedPassword.ToString().Substring(0, returnLength);
               }

return "";
}
```

ResetDisplayListTrigger

Occurs when the Context Server receives an execution of the following method:

- *ResetDisplayList, page 62*

Syntax

Event ResetDisplayListTrigger()

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select resets its worklist to be empty on receiving this event. Display does nothing.

IMPAX 6.2 clears the contents of the Image area on receiving this event.

ScreenAttributeChange

Occurs when the Context Server receives changes to any of following properties:

- *NumberOfXMonitors, page 68*
- *NumberOfYMonitors, page 68*
- *ScreenToCentreOn, page 69*

Syntax

Event ScreenAttributeChange(*number_of_x_monitors***,** *number_of_y_screens***,** *screen_to_center_on***)**

Part	Data Type	Description
number_of_x_monitors	Long	The number of monitors horizontally.
number_of_y_monitors	Long	The number of monitors vertically — always 1.
screen_to_center_on	Long	Which screen to center the form on.

IMPAX Behavior

IMPAX does nothing on receiving this event.

Notes

IMPAX enumerates the monitors as follows:



number_of_x_monitors = 1

number_of_y_monitors = 1

screen_to_center_on = 1



number_of_x_monitors = 2

number_of_y_monitors = 1

screen_to_center_on = 1 or 2



number_of_x_monitors = 3

number_of_y_monitors = 1

screen_to_center_on = 1, 2, or 3



number_of_x_monitors = 4

number_of_y_monitors = 1

screen_to_center_on = 1, 2, 3, or 4

ShowDisplayListTrigger

Occurs when the Context Server receives an execution of the following method:

- *ShowDisplayList, page 63*

Syntax

Event ShowDisplayListTrigger()

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Display comes up showing the current contents of display list on receiving this event.

IMPAX 6.2 opens the Image area if there are studies in the cycle list or in a patient tab in the bottom navigation bar.

StudyIdentifierChange

Occurs when the Context Server receives a change to the following property:

- *StudyId, page 70*

Syntax

Event StudyIdentifierChange(ByVal *study_id* **As String)**

Part	Data type	Description
study_id	String	An identifier that RIS and PACS systems agree on as a study (or exam) identifier. If set by the IMPAX Client, this represents the DICOM study UID.

IMPAX behavior

IMPAX does nothing on receiving this event.

StudyReadTrigger

Occurs when the Context Server receives an execution of the following method:

- *StudyRead, page 63*

Syntax

Event StudyReadTrigger()

IMPAX behavior

IMPAX does nothing on receiving this event.



Tip:

Use the values stored in the property StudyID in the Context Server to write code in this event to manage the fact that this study was read.

StudyStatusChange

Occurs when the Context Server receives a change of the following property:

- *StudyStatus, page 70*

Syntax

Event StudyStatusChange(ByVal *study_status* **As String)**

IMPAX behavior

IMPAX does nothing on receiving this event.

UserNameChange

Occurs when the Context Server receives a change to the following property:

- *UserName, page 71*

Syntax

Event UserNameChange(ByVal *User_name* **As String)**

Part	Data Type	Description
User_name	String	The user ID to be used for client applications that support user authentication.

IMPAX behavior

IMPAX does nothing on receiving this event.

Methods Reference

9

This chapter describes the Context Server methods. Some methods are called only by the integrated application.

AddToDisplayList

Call this method to cause the Context Server to raise the *AddToDisplayListTrigger* event, page 48, in all registered applications.

Syntax

Object.**AddToDisplayList**

Part	Description
Object	The Context Server object.

IMPAX behavior

IMPAX does not call this method.



Note:

Before calling this method, set the StudyId property of the DICOM study UID of the study to be displayed.

DoAction

Call this method to have the Context Server raise the corresponding *Action* event in all registered applications. Refer to Chapter 7: *SYNC_DISPLAY*.

ALaunchApplication

Call this method to have the Context Server raise the *LaunchApplicationTrigger* event, page 51, in all registered applications.

Syntax

Object.**LaunchApplication**

Part	Description
Object	The Context Server object.

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select can call this method through a script button.

IMPAX 6.2 does not call this method.

PerformUserValidation

Call this method to cause the Context Server to raise the *PerformUserValidationTrigger* event, page 52, in all registered applications.

Syntax

Object.**PerformUserValidation**

Part	Description
Object	The Context Server object.

IMPAX behavior

IMPAX calls this method when a user logs in. For more information, refer to *PerformUserValidationTrigger, page 52*.

ResetDisplayList

Call this method to cause the Context Server to raise the *ResetDisplayListTrigger* event, page 57, in all registered applications.

Syntax

Object.**ResetDisplayList**

Part	Description
Object	The Context Server object.

IMPAX Behavior

IMPAX does not call this method.

ShowDisplayList

Call this method to cause the Context Server to raise the *ShowDisplayListTrigger* event, page 59, in all registered applications.

Syntax

Object.**ShowDisplayList**

Part	Description
Object	The Context Server object.

IMPAX behavior

IMPAX does not call this method.



Note:

This method works only for studies that are cached.

StudyRead

Call this method to cause the Context Server to raise the *StudyReadTrigger* event, page 59, in all registered applications.

Syntax

Object.**StudyRead**

Part	Description
Object	The Context Server object.

IMPAX behavior

IMPAX does not call this method.



Tip:

Before calling this method, populate the PatientID and StudyID properties to identify the patient and study that has been read.

Properties Reference 10

This chapter describes the Context Server properties.

AccessionNumber

Use this property to set the accession number.

Setting this property raises the *AccessionNumberChange* event, described on page 48. If the *PatientId* (page 69) and *BrokerDSN* (page 65) properties are not null, then the *StudyId* property (page 70) is retrieved from the RIS Gateway or PACS broker using a stored procedure call.

Syntax

Object.**AccessionNumber** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	String	A value that identifies an accession number that both the PACS and RIS applications agree on.

IMPAX behavior

IMPAX sets the accession number into this property each time a new study is opened for viewing. If more than one study appears on-screen, the accession number for the study that has focus is put in the Context Server. In a multi-monitor configuration in which a study is displayed on each monitor, this property is multi-valued and contains the accession numbers of all displayed studies. When multiple values are used, each is delimited by a backslash character.

AlwaysOnTopFlag

Use this property to set a flag in the Context Server that an application should present itself as “always on top”; that is, as the topmost window.

Setting this property raises the *AlwaysOnTopChange* event, described on page 49.

Syntax

Object.**AlwaysOnTopFlag** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	Long	0 = the application is not always on top. 1 = the application is always on top.

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select sets this property to -1.

IMPAX 6.2 sets this property to 0.

ApplicationName

Use this property to set the name of the EXE or application name to launch.

Setting this property raises the *ApplicationNameChange* event, described on page 49.

Syntax

Object.**ApplicationName** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	String	Value that identifies the path and file name of the application to launch.

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select can set this property through a Script button.

IMPAX 6.2 does not set this property.

BrokerDSN

In IMPAX 6.2, Client computers no longer have direct ODBC connections to RIS Gateway or PACS brokers. As a result, the BrokerDSN property can no longer be used to look up the Study UID when only the PatientID and AccessionNumber are available. Instead, use the GETUIDBYPIDACN *DoAction* method, page 62, to achieve this result.

Setting this property does not cause any event to be raised.

Syntax

Object.**BrokerDSN** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.

Part	Data type	Description
Value	String	Any string that defines the name of the ODBC DSN that points at the RIS Gateway or PACS broker (not applicable to IMPAX 6.2).

IMPAX behavior

IMPAX does not set this property.



Note:

In IMPAX 6.2, Client computers no longer have direct ODBC connections to RIS Gateway or PACS broker.

ContextServer

Creates an instance of the Context Server inside an application.

Syntax

Object = *Connector*.**ContextServer**

Part	Data type	Description
Object	Object	The Context Server object.
Connector	Connector_class	A previously created connector class object.

This property is read-only and is used to provide a reference to the internal Context Server class.

FontName

Use this property to identify what font the application should use for user interface text.

Setting this property raises the *FontChange* event, described on page 49.

Syntax

Object.**FontName** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	String	Value that identifies the font face to use (such as Arial, Courier New, or Tahoma.)

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select sets this property at startup or if the font name is changed during execution.

IMPAX 6.2 sets this property to Tahoma.

FontSize

Use this property to identify what size font the application should use for interface text.

Setting this property raises the *FontChange* event, described on page 49.

Syntax

Object.**FontSize** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	Long	Value that identifies the point size of the font to use.

IMPAX Behavior

In IMPAX 4.5 and 5.*x*, Select sets this property at startup or if the font size is changed during execution.

IMPAX 6.2 sets this property to 11.

IconSize

Use this property to identify what size tool buttons the application uses.

Setting this property raises the *IconSizeChange* event, described on page 50.

Syntax

Object.**IconSize** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	String	Value that identifies what size icons to use. **IMPAX 6.2:** SMALL = 22x22 pixel 16-color bitmap MEDIUM = 42x42 pixel 16-color bitmap **IMPAX 4.5 and 5.*x*:** SMALL = 40x40 pixel 16-color bitmap MEDIUM = 60x60 pixel 16-color bitmap LARGE = 80x80 16-color bitmap

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select sets this property at startup or if the icon size is changed during execution.

IMPAX 6.2 sets this property to SMALL.

NumberOfXMonitors

Use this property to set the number of monitors that are present horizontally.

Setting this property raises the *ScreenAttributeChange* event, described on page 58.

Syntax

Object.**NumberOfXMonitors** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	Long	Value that represents the number of monitors horizontally.

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select sets this property at startup.

IMPAX 6.2 sets this property to the number of monitors for the Image area.

NumberOfYMonitors

Use this property to set the number of monitors that are present vertically.

Setting this property raises the *ScreenAttributeChange* event, described on page 58.

Syntax

Object.**NumberOfYMonitors** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	Long	Value that represents the number of monitors vertically.

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select sets this property at startup.

IMPAX 6.2 sets this property to 1, the only value supported in IMPAX 6.2.

Password

Use this property to set the password for the application. This property is typically used to perform user authentication on the RIS or PACS software.

Setting this property raises the *PasswordChange* event, described on page 51.

Syntax

Object.**Password** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	String	Value that identifies the password that corresponds to the name of the user stored in the property *UserName, page 71*.

IMPAX behavior

IMPAX sets this property when a user logs in, and clears it when a user logs out. The password is encrypted as described in the *PerformUserValidationTrigger* event section, page 52. The password value returned by this property is the unencrypted value.

In IMPAX 4.5 and 5.x, Select can also set this property through a Script button.

PatientId

Use this property to set the patient-level identifier.

Setting this property raises the *PatientIdentifierChange* event, described on page 51. If the *AccessionNumber* (page 64) and *BrokerDSN* (page 65) properties are not null, then the *StudyId* property (page 70) will be retrieved from the RIS Gateway or PACS broker using a stored procedure call.

Syntax

Object.**PatientId** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	String	Value that identifies a patient level identifier that both PACS and RIS applications agree on.

IMPAX behavior

IMPAX sets this each time a new patient is opened for viewing.

In IMPAX 4.5 and 5.*x*, Select can also set this through a Script button.

ScreenToCentreOn

Use this property to identify which monitor the application should be shown on.

Setting this property raises the *ScreenAttributeChange* event, described on page 58.

Syntax

Object.**ScreenToCentreOn** = *Value*

Part	Data type	Description
Object	Object	The Context Server object.
Value	Long	Value that represents which monitor the application should be centered on.

IMPAX behavior

In IMPAX 4.5 and 5.*x*, Select sets this property at startup.

IMPAX 6.2 sets this property to the monitor that is showing the List and Text areas.

StudyId

Use this property to set the study-level (or exam-level) identifier.

Setting this property raises the *StudyIdentifierChange* event, described on page 59. Setting this property also clears the *AccessionNumber* property, page 64.

Syntax

Object.**StudyId** = *Value*

Part	Data Type	Description
Object	Object	The Context Server object.
Value	String	Value that identifies a study-level identifier that both PACS and RIS applications agree on.

IMPAX behavior

IMPAX sets this using the study's DICOM study UID each time a new study is opened for viewing. If more than one study appears on-screen, the study UID for the study that has focus is put in the Context Server. In a multi-monitor situation in which a study is displayed on each monitor (and no study has focus), this property is multi-valued and contains the Study UIDs of all displayed studies, delimited by backslashes (\).



Note:

Most applications expect a Study UID to be in this field rather than a Study ID. The Accession Number field should be used when Study UID is not available or appropriate.

StudyStatus

Use this property to set a string in the Context Server to indicate the status of the study specified by the *StudyId* property, page 70.

Setting this property raises the *StudyStatusChange* event, described on page 60.

Syntax

Object.**StudyStatus** = *Value*

Part	Data Type	Description
Object	Object	The Context Server object.
Value	String	Any string that both applications will understand that indicates the status of the study.

IMPAX behavior

IMPAX sets this property each time a new study is opened for viewing. If more than one study appears on-screen, the status for the study that has focus is put in the Context Server. In a multi-monitor situation where a study is displayed on each monitor (and no study has focus), this property is multi-valued and contains the status of all displayed studies, delimited by backslashes (\).

Valid IMPAX study status values include:

Status	Code	Button appearance
New	N	
Dictation in progress	d	
Trainee dictation in progress	t	
Dictated	D	
Trainee dictated	T	
Reported	r	
Approved	R	

UserName

Use this property to set a user ID for the application. This property is typically used to perform user authentication on the RIS or PACS software.

Setting this property raises the *UserNameChange* event, described on page 60.

Syntax

Object.**UserName** = *Value*

Part	Data Type	Description
Object	Object	The Context Server object.

Part	Data Type	Description
Value	String	Value that identifies the user.

IMPAX behavior

IMPAX sets this property when a user logs in, and clears it when a user logs out.

In IMPAX 4.5 and 5.*x*, Select can also set this property through a Script button.

Appendix A: IMPAX 6.2 Script Interfaces

A

The interfaces listed here are current as of IMPAX 6.2.

1 IScriptApplication

```
/// <summary>
/// context server attributes that we can get/set
/// </summary>
public enum ContextServerAttributes
{
                PatientId = 0,
                StudyUid = 1,
                AccessionNumber = 2,
                StudyStatus = 3,
                UserName = 4,
                Password = 5
}

/// <summary>
/// The IScriptApplication interface contains the
/// definitions of the scriptable aspects of the
/// application, or the application-level components,
/// such as the context server.
/// These methods will execute on a separate thread from
/// the main GUI.
/// </summary>
public interface IScriptApplication
```

```
{
                /// <summary>
                /// get a value from the context server.
                /// Example:
GetValueFromContextServer(ContextServerAttributes.StudyUid)
                /// </summary>
                /// <param name="contextServerAttribute">one of the
/// ContextServerAttributes values</param>
                /// <returns>the value of the item in the specified
context server
                /// attribute</returns>
                string
GetValueFromContextServer(ContextServerAttributes
contextServerAttribute);


                /// <summary>
                /// set a value into the context server.
                /// Example:
SetValueIntoContextServer(ContextServerAttributes.PatientId,
                ///   "A123")
                /// </summary>
                /// <param name="contextServerAttribute">one of the
                /// ContextServerAttributes values</param>
                /// <param name="valueToSet">the value to set into
the indicated
                /// context server attribute</param>
                void
SetValueIntoContextServer(ContextServerAttributes
                      contextServerAttribute, string valueToSet);


                /// <summary>
                /// execute a context server DoAction command.
                /// Example:
ContextServerDoAction("START_DICTATION", "", "", "")
                /// </summary>
                /// <param name="actionName"></param>
                /// <param name="paramTwo"></param>
                /// <param name="paramThree"></param>
                /// <param name="paramFour"></param>
                void ContextServerDoAction(string actionName,
string paramTwo,
                      string paramThree, string paramFour);
```

```
                /// <summary>
                /// associate a keyword.
                /// Example:
AssociateKeywordWithStudyUid("1.2.3.4", "NORMAL CHEST")
                /// </summary>
                /// <param name="studyUid">the study_uid of the
study we want to associate</param>
                /// <param name="keyword">the keyword to associate
with the study</param>
                /// <returns>true if the operation was successful,
false otherwise</returns>
                bool AssociateKeywordWithStudyUid(string studyUid,
string keyword);


                /// <summary>
                /// associate a keyword.
                /// Example: AssociateKeywordWithPidAcn("P123",
"A456", "NORMAL CHEST")
                /// </summary>
                /// <param name="patientId">the patient_id of the
study we want
/// to associate</param>
                /// <param name="accessionNumber">the
accession_number of the study
/// we want to associate</param>
                /// <param name="keyword">the keyword to associate
with the study</param>
                /// <returns>true if the operation was successful,
false otherwise</returns>
                bool AssociateKeywordWithPidAcn(string patientId,
string
                      accessionNumber, string keyword);


                /// <summary>
                /// associate a keyword.
                /// Example: AssociateKeywordWithAcn("A456",
"NORMAL CHEST")
                /// </summary>
                /// <param name="accessionNumber">the
accession_number of the study
                /// we want to associate</param>
                /// <param name="keyword">the keyword to associate
with the study</param>
```

```
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool AssociateKeywordWithAcn(string
accessionNumber, string keyword);


        /// <summary>
        /// transmit a study.
        /// Example: TransmitStudyWithStudyUid("1.2.3.4",
"MYWEBSERVER")
        /// </summary>
        /// <param name="studyUid">the study_uid of the
study we want
        /// to transmit</param>
        /// <param name="aeTitle">the AETitle of the
machine we want to
        /// send the study to</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool TransmitStudyWithStudyUid(string studyUid,
string aeTitle);


        /// <summary>
        /// transmit a study.
        /// Example: TransmitStudyWithPidAcn("P123",
"A456", "MYWEBSERVER")
        /// </summary>
        /// <param name="patientId">the patient_id of the
study we want
        /// to transmit</param>
        /// <param name="accessionNumber">the
accession_number of the study
        /// we want to transmit</param>
        /// <param name="aeTitle">the AETitle of the
machine we want to
        /// send the study to</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool TransmitStudyWithPidAcn(string patientId,
string
              accessionNumber, string aeTitle);


        /// <summary>
        /// transmit a study.
```

```
        /// Example: TransmitStudyWithAcn("A456",
"MYWEBSERVER")
        /// </summary>
        /// <param name="accessionNumber">the
accession_number of the study
        /// we want to transmit</param>
        /// <param name="aeTitle">the AETitle of the
machine we want to
        /// send the study to</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool TransmitStudyWithAcn(string accessionNumber,
string aeTitle);


        /// <summary>
        /// fire an application event
        /// </summary>
        /// <param name="appEvent">an
ApplicationEventArgs</param>
        void
FireApplicationEventFromScript(AgfaHC.Framework.ApplicationEventAr
gs appEvent);
}
```

IScriptDisplay

```
/// <summary>
/// The IScriptDisplay interface contains the
/// definitions of the scriptable aspects of the
/// image view area.
/// These methods will execute on a separate thread from
/// the main GUI.
/// </summary>
public interface IScriptDisplay
{
        /// <summary>
        /// blank the image view area.
        /// Example: ClearDisplay()
        /// </summary>
        void ClearDisplay();
```

```
        /// <summary>
        /// bring a study forward in the Image area.
        /// Example:
ActivateStudyInDisplayWithStudyUid("1.2.3.4")
        /// </summary>
        /// <param name="studyUid">the study_uid of the
study we want to see</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool ActivateStudyInDisplayWithStudyUid(string
studyUid);


        /// <summary>
        /// bring a study forward in the image view area.
        /// Example:
ActivateStudyInDisplayWithPidAcn("P123", "A456")
        /// </summary>
        /// <param name="patientId">the patient_id of the
study we
        /// want to see</param>
        /// <param name="accessionNumber">the
accession_number of the
        /// study we want to see</param>
        /// <returns>true if the operation was successful,
false
        /// otherwise</returns>
        bool ActivateStudyInDisplayWithPidAcn(string
patientId, string accessionNumber);


        /// <summary>
        /// bring a study forward in the image view area.
        /// Example: ActivateStudyInDisplayWithAcn("A456")
        /// </summary>
        /// <param name="accessionNumber">the
accession_number of the study
/// we want to see</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool ActivateStudyInDisplayWithAcn(string
accessionNumber);


        /// <summary>
```

```
        /// remove a study from the image view area.
        /// Example:
RemoveStudyFromDisplayWithStudyUid("1.2.3.4")
        /// </summary>
        /// <param name="studyUid">the study_uid of the
study we want
/// to remove</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool RemoveStudyFromDisplayWithStudyUid(string
studyUid);


        /// <summary>
        /// remove a study from the image view area.
        /// Example:
RemoveStudyFromDisplayWithPidAcn("P123", "A456")
        /// </summary>
        /// <param name="patientId">the patient_id of the
study we want
/// to remove</param>
        /// <param name="accessionNumber">the
accession_number of the study
/// we want to remove</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool RemoveStudyFromDisplayWithPidAcn(string
patientId, string accessionNumber);


        /// <summary>
        /// remove a study from the image view area.
        /// Example: RemoveStudyFromDisplayWithAcn("A456")
        /// </summary>
        /// <param name="accessionNumber">the
accession_number of the study
/// we want to remove</param>
        /// <returns>true if the operation was successful,
false otherwise</returns>
        bool RemoveStudyFromDisplayWithAcn(string
accessionNumber);


        /// <summary>
        /// add the study to the image view area.
        /// Example: AddToDisplayListWithAcn("A456")
```

```
        /// </summary>
        /// <param name="accessionNumber">the accession
number of the study
/// we want to add</param>
        void AddToDisplayListWithAcn(string
accessionNumber);

        /// <summary>
        /// add the study to the image view area.
        /// Example:
AddToDisplayListWithStudyUid("1.2.3.4")
        /// </summary>
        /// <param name="studyUid">the study uid of the
study we want to add</param>
        void AddToDisplayListWithStudyUid(string studyUid);

        /// <summary>
        /// add the study to the image view area.
        /// Example: AddToDisplayListWithPidAcn("P123",
"A456")
        /// </summary>
        /// <param name="patientId">the patient id of the
study we
        /// want to add</param>
        /// <param name="accessionNumber">the accession
number of the study
        /// we want to add</param>
        void AddToDisplayListWithPidAcn(string patientId,
string accessionNumber);

/// <summary>
        /// add the study to the image view area.
        /// Example:
AddToDisplayListWithGlobalPidAcn("GP123", "A456",
/// StudyOpenActions.CycleList)
        /// </summary>
        /// <param name="gpatientId">the global patient id
of the study we
        /// want to add</param>
        /// <param name="accessionNumber">the accession
number of the study
        /// we want to add</param>
```

```
                /// <param name="openAction">the way to open the
study</param>
                void AddToDisplayListWithGlobalPidAcn(string
gpatientId, string accessionNumber,
StudyOpenActions openAction);
```

IScriptListArea

```
/// <summary>
/// ways we can sort a worklist
/// </summary>
public enum WorkListSortDirections
{
                Ascending = 0,
                Descending = 1
}

/// <summary>
/// The IScriptListArea interface contains the
/// definitions of the scriptable aspects of the
/// list area.
/// These methods will execute on a separate thread from
/// the main GUI.
/// </summary>
public interface IScriptListArea
{
                /// <summary>
                /// refresh the list area worklists.
                /// Example: RefreshWorkLists()
                /// </summary>
                void RefreshWorkLists();

                /// <summary>
                /// reset the adhoc search criteria.
                /// Example: ResetSearchCriteria()
                /// </summary>
                void ResetSearchCriteria();

                /// <summary>
```

```
                /// get the indicated value from the selected row
in the current
                /// worklist.
                /// Example: GetValueFromWorkList("study",
"accession_number")
                /// </summary>
                /// <param name="attributeClass">the attribute
class of the item
                /// </param>
                /// <param name="attributeName">the attribute name
of the item
                /// </param>
                /// <returns>the value of the item specified</
returns>
                string GetValueFromWorkList(string attributeClass,
string
                      attributeName);

                /// <summary>
                /// get the IWorkListItem from the selected row in
the current
                /// worklist.
                /// Example: PacsStudy study = GetWorkListItem()
                /// </summary>
                /// <returns>an IWorkListItem for the selected row
in the current
                /// worklist</returns>
                IWorkListItem GetWorkListItem();

                /// <summary>
                /// sort the current worklist.
                /// Example: SortWorkList("study", "study_date",
/// WorkListSortDirections.Ascending)
                /// </summary>
                /// <param name="attributeClass">the attribute
class of the item</param>
                /// <param name="attributeName">the attribute name
of the item</param>
                /// <param name="sortDirection">a
WorkListSortDirections value
/// (ascending or descending)</param>
                void SortWorkList(string attributeClass, string
attributeName,
                      WorkListSortDirections sortDirection);
```

```
                /// <summary>
                /// clear the ad hoc finder.
                /// Example: ClearSearchWorkList()
                /// </summary>
                void ClearSearchWorkList();

                /// <summary>
                /// add a study to the search worklist.
                /// Example:
AddToSearchWorkListWithStudyUid("1.2.3.4")
                /// </summary>
                /// <param name="studyUid">the study_uid of the
study we want to
                /// add</param>
                void AddToSearchWorkListWithStudyUid(string
studyUid);

                /// <summary>
                /// add a study to the search worklist.
                /// Example: AddToSearchWorkListWithPidAcn("P123",
"A456")
                /// </summary>
                /// <param name="patientId">the patient_id of the
study we want
                /// to add</param>
                /// <param name="accessionNumber">the
accession_number of the study
                /// we want to add</param>
                void AddToSearchWorkListWithPidAcn(string
patientId, string
                      accessionNumber);

                /// <summary>
                /// add a study to the search worklist.
                /// Example: AddToSearchWorkListWithAcn("A456")
                /// </summary>
                /// <param name="accessionNumber">the
accession_number of the study
                /// we want to add</param>
                void AddToSearchWorkListWithAcn(string
accessionNumber);
```

```
        /// <summary>
        /// display the current worklist.
        /// Example: DisplayWorkList()
        /// </summary>
        void DisplayWorkList();
}
```

Attributes available for each worklist type

The specific attributes available for each worklist type for the methods that return data from the List area are listed below.

The column containing the referenced data must be included in the worklist, or the data will not be returned. For example, if body_part is referenced, the body part column must be included in the worklist.

Standard worklists

Standard worklists are populated based on defined search criteria.

Class	Name
ae	ae_title
patient	current_patient_location
patient	global_patient_id
patient	other_patient_ids
patient	patient_birth_date
patient	patient_id
patient	patient_link
patient	patient_name
patient	patient_name_id_utf8
patient	patient_name_ph_utf8
patient	patient_name_sb_utf8
patient	patient_sex
series	manufacturer
study	accession_number
study	additional_patient_history
study	admitting_diagnosis_description
study	best_location
study	body_part
study	body_part_examined
study	branch_of_service

Class	Name
study	confidentiality_code
study	date_time_verified
study	department_visit_number
study	institution_name
study	institutional_department_name
study	is_summary_study
study	lossy_compressed_status
study	military_rank
study	modality
study	name_of_physicians_reading_study
study	num_conflicts
study	num_images
study	num_objects
study	operator_name
study	patient_age
study	performing_physician_id_utf8
study	performing_physician_name
study	performing_physician_ph_utf8
study	performing_physician_sb_utf8
study	performing_physician_utf8
study	reason_for_study
study	reason_for_study_utf8
study	referring_physician_id_utf8
study	referring_physician_name
study	referring_physician_ph_utf8
study	referring_physician_sb_utf8
study	referring_physician_utf8
study	requested_procedure_code
study	requested_procedure_id
study	requesting_physician
study	requesting_physician_id_utf8
study	requesting_physician_ph_utf8
study	requesting_physician_sb_utf8
study	requesting_physician_utf8
study	requesting_service
study	specialty

Class	Name
study	station_name
study	status
study	study_comments
study	study_comments_utf8
study	study_date
study	study_description
study	study_description_utf8
study	study_id
study	study_patient_location
study	study_priority_id
study	study_ref
study	study_status_id
study	study_time
study	study_uid
study_keyword	keywords
study_location	protected
study_location_view	age_at_study
study_location_view	num_cache_objects
study_location_view	num_cdr_objects
study_location_view	num_dvd_objects
study_location_view	num_hsm_objects
study_location_view	num_mod_objects
study_location_view	num_offline_objects
study_location_view	num_pacs_objects
study_location_view	num_tape_objects
study_location_view	num_web_objects
study_procedure_info	scheduled_procedure_step_id

Scheduled worklists

Scheduled worklists contain date attributes, and are used to assemble studies for rounds, conferences, and clinics.

Class	Name
patient	current_patient_location
patient	other_patient_ids
patient	patient_birth_date
patient	patient_id

Class	Name
patient	patient_name
patient	patient_name_id_utf8
patient	patient_name_ph_utf8
patient	patient_name_sb_utf8
patient	patient_sex
study	accession_number
study	best_location
study	branch_of_service
study	confidentiality_code
study	date_time_verified
study	department_visit_number
study	institution_name
study	institutional_department_name
study	lossy_compressed_status
study	military_rank
study	modality
study	name_of_physicians_reading_study
study	num_images
study	num_objects
study	patient_age
study	performing_physician_id_utf8
study	performing_physician_name
study	performing_physician_ph_utf8
study	performing_physician_sb_utf8
study	reason_for_study
study	referring_physician_id_utf8
study	referring_physician_name
study	referring_physician_ph_utf8
study	referring_physician_sb_utf8
study	requesting_physician
study	requesting_physician_id_utf8
study	requesting_physician_ph_utf8
study	requesting_physician_sb_utf8
study	requesting_service
study	specialty
study	station_name

Class	Name
study	status
study	study_comments
study	study_date
study	study_description
study	study_description_utf8
study	study_patient_location
study	study_priority_id
study	study_ref
study	study_time
study	study_uid
study_keyword	keywords
study_location_view	age_at_study
study_location_view	num_cache_objects
study_location_view	num_cdr_objects
study_location_view	num_dvd_objects
study_location_view	num_hsm_objects
study_location_view	num_mod_objects
study_location_view	num_offline_objects
study_location_view	num_pacs_objects
study_location_view	num_tape_objects
study_location_view	num_web_objects
study_procedure_info	requested_procedure_id
study_procedure_info	scheduled_procedure_step_id

Sign off worklists

Sign off worklists assembles reports to review and sign off, when the IMPAX Client is integrated with an Agfa RIS.

Class	Name
report	accession_number
report	addressee
report	conclusions
report	department
report	interpretation_transcribed_date
report	interpretation_transcribed_time
report	order_ref
report	order_referring_physician

Class	Name
report	patient_birth_date
report	patient_other_names
report	patient_ref
report	patient_sex
report	reason_for_study
report	report_author
report	report_ref
report	report_status
report	report_uid
report	responsible_physician
report	study_date
report	study_description
report	study_instance_uid
report	study_ref
report	study_time
report	transcriber

Teaching files

Advanced Search can be used to find teaching files by file or folder name or by comments. The search results area can be customized for displaying the teaching files.

Class	Name
mitra_folder	owner_id
mitra_folder_item	folder_ref
mitra_folder_item	item_name
mitra_folder_item	item_ref
mitra_folder_item	tf_comment
patient	current_patient_location
patient	patient_id
patient	patient_name
study	accession_number
study	body_part
study	confidentiality_code
study	date_time_verified
study	institution_name
study	institutional_department_name
study	lossy_compressed_status

Class	Name
study	modality
study	num_images
study	num_objects
study	patient_age
study	patient_birth_date
study	patient_history
study	patient_id
study	patient_name
study	patient_sex
study	reason_for_study
study	referring_physician_id_utf8
study	referring_physician_name
study	referring_physician_ph_utf8
study	referring_physician_sb_utf8
study	referring_physician_utf8
study	requesting_physician
study	requesting_physician_id_utf8
study	requesting_physician_ph_utf8
study	requesting_physician_sb_utf8
study	requesting_physician_utf8
study	specialty
study	station_name
study	status
study	study_comments
study	study_date
study	study_description
study	study_patient_location
study	study_priority_id
study	study_ref
study	study_time
study	study_uid
study_location_view	age_at_study
study_location_view	num_cache_objects
study_location_view	num_cdr_objects
study_location_view	num_dvd_objects
study_location_view	num_hsm_objects

Class	Name
study_location_view	num_mod_objects
study_location_view	num_offline_objects
study_location_view	num_pacs_objects
study_location_view	num_tape_objects
study_location_view	num_web_objects

Sample script

This sample script reads the patient ID and accession number values from the context server, then builds a URL and calls it using the default system browser.

The script code must be contained in a VB.NET module named “Script”, and the script must contain a Main() method, which will be called by the IMPAX 6.2 application.

```
Option Strict Off

Imports System
Imports System.Windows.Forms

' the module must be named Script

Module Script
' the IMPAX application will call the Main method

Sub Main()
                    dim patientId as string
                    dim accessionNumber as string
                    dim url as string

                    ' get the patient id and accession number
from
‘ the context server
                    ' patient id == 0
                    ' accession number == 2

patientId = ScriptApplication.GetValueFromContextServer(0)
accessionNumber = ScriptApplication.GetValueFromContextServer(2)

                    ' build a url
```

```
                    url = "http://ris/ShowOrder.aspx?patientId="
+ _
patientId + "&accessionNumber=" + accessionNumber

' show the url

                    System.Diagnostics.Process.Start(url)
    End Sub
End Module
```

Index

A

accession number, setting 64
AccessionNumber property 64
    opening studies 22
AccessionNumberChange event 48
    opening studies 22
Action event 48
    bringing a study forward 24–25
    canceling dictation 31
    clearing the Image area 23
    ending dictation 28, 31
    hiding List and Text areas 21
    logging out 18
    rolling back dictation status 43–44
    running scripts 43
    setting window state 44
    specifying studies 40
    starting and ending dictation 29
    starting dictation 28
    starting dictations 39
    starting multiple-study dictation 29
    updating study status 46
Action events
    completing dictations 39
activating
    List area 34
active worklist. *See* Configure area, configuring dictation
ADD_TO_DISPLAY_LIST 36
ADD_TO_DISPLAY_LIST event
    adding studies to a worklist 20
ADD_TO_DISPLAY_LIST_WITH_PRIORS 37
adding studies and priors to cycle list 37
adding studies to a worklist 20, 34
adding studies to cycle list 36
AddToDisplayList method 61
    opening studies 21
AddToDisplayListTrigger event 48
    opening studies 21
AE title attribute in worklists 84
AlwaysOnTopChange event 49
AlwaysOnTopFlag property 64
ApplicationName property 65
ApplicationNameChange event 49
approved studies 27, 71
architecture, Context Server 9
attributes of worklist types 84–91
authenticating users 68, 71

B

BLANK_DISPLAY 38
BLANK_DISPLAY action 23
blanking out Image area. *See* clearing the Image area
bringing a study forward 24, 45
Broker connection 65
BrokerDSN property 65
button size 50, 67

C

cached studies 63
canceling dictation 31
changing
    password 51
    study status 31
    user name 60
classes of Context Server 9
clearing the Image area 21, 23, 38, 58
    script example 34
code sample
    encrypting passwords 52
    setting study UID 20
column customization, worklist 84
COMPLETE_DICTATION action 31
completing dictation 28, 31
    multiple studies 30
    two-state mode 28
completing dictations 39
connector class 9
context server class 9
Context Server definition 7
ContextServer property 66
creating scripts 33
cycle list 20
    adding studies to 21–22, 49

D

database of integrated application 20

dictating studies 27–30
dictation application, launching..... 26
dictation mode.....27–29
dictation status 26
DICTATION_COMPLETED..... 39
DICTATION_COMPLETED action 28–29, 31
DICTATION_STARTED..... 38
DICTATION_STARTED action 28–30
dictations completing 39
dictations starting..... 38
displaying studies 21, 30, 45
script example 34
DoAcrtion
specifying available studies..... 40
DoAction
adding studies and priors to cycle list..... 37
adding studies to cycle list 36
bringing a study forward 45
clearing Image area 38
completing dictations 39
getting DICOM Study UID..... 40
getting list of DICOM Study UID 40
hiding List and Text Areas..... 41
LOGOUT from Integrated Application..... 41
REMOVE_ALL_FROM_CYCLE_LIST 42
REMOVE_ALL_FROM_WORKLIST 42
rolling back dictation status 43
running scripts..... 43
setting window state..... 44
starting dictation 38
taking focus 44
updating study status..... 46
DoAction method..... 61
bringing a study forward 24–25
canceling dictation 31
clearing the Image area..... 23
ending dictation 28, 31
hiding List and Text areas 21
identifying studies 19
logging out 18
running a script 33
starting and ending dictation 28
starting dictation 30
DoAction methods and events reference ...36–47

E

encrypting passwords 52, 69
ending dictation.....28, 31
multiple studies 30
two-state mode..... 28
events reference.....48–60
example script
see also script interfaces
add studies to Image area 19, 34
build URL 35
clear Image area and activate List area.... 34
display search worklist..... 34

F

folder attributes, teaching files 89
FontChange event 49
FontName property..... 66
FontSize property..... 67

G

GETSTUDIESINDISPLAY..... 39
GETUIDBYPICACN..... 40
GETUIDSBYPID..... 40
GUI
button size 67
text 66

H

HIDE_SELECT..... 40
HIDE_SELECT action 21
hiding
Image area 44
List and Text areas 21
hiding List and Text areas..... 40

I

IconSize property 67
IconSizeChange event 50
identifying studies..... 19
Image area
bringing a study forward..... 24
bringing a study foward 45
clearing.....21, 23, 38, 58
cycling studies..... 49, 59
IMPAX Client startup..... 12
IMPAX Context Server. *See* Context Server definition
in progress studies..... 26, 71
interface, script 73–91
IScriptDisplay..... 77
IScriptListArea 81–84

L

LaunchApplication method 62
LaunchApplicationTrigger event 51
launching dictation application 26
launching programs 49, 65
List area
hiding 21
specifying monitor 70
worklist types..... 84
logging into

IMPAX 62
IMPAX and an integrated application. 12–17
logging out from IMPAX logging out from IMPAX 41
logging out from the Integrated Application... 41
logging out of
IMPAX and an integrated application. 17–18
LOGOUT action 18
LOGOUT from IMPAX Client 41

M

methods reference 61–63
Microsoft .NET Framework 33
minimizing List and Text area 44
mode of dictation 27–29
monitors
List area 69
numbering 58, 68
multi-monitor configurations 64, 70–71
multiple-study dictation 29

N

name of font 50, 66
.NET Framework 33
new features 9
new studies 26
non-integrated reporting systems 27
NumberOfXMonitors property 68
setting on startup 12
NumberOfYMonitors property 68
setting on startup 12

O

ODBC DSN 65
opening studies 21

P

PACS broker 65
Password property 68
setting on startup 12
single sign-off 17–18
single sign-on 13–16
PasswordChange event 51
sending on startup 12
single sign-off 17–18
single sign-on 13–16
patient attributes
scheduled worklists 86
standard worklist 84
patient attributes, teaching files 89
PatientId property 69
opening studies 22
PatientIdentifierChange event 51
opening studies 22
PerformUserValidation method 62
single sign-on 13–16
PerformUserValidationTrigger event 52
single sign-on 13–16
point size of font 50
properties
reference 64–72
set on IMPAX startup 12
public classes 9

R

REMOVE_ALL_FROM_CYCLE_LIST 42
REMOVE_ALL_FROM_WORKLIST 42
removing studies from the cycle list 42
report attributes, sign off worklists 88
reporting application, launching 26
reporting status 26
ResetDisplayList method 62
clearing the Image area 21, 24
ResetDisplayListTrigger event 57
clearing the Image area 21, 24
RIS integrations 68
ROLLBACK_IN_PROGRESS_STUDY 43
RUN_SCRIPT 43
RUN_SCRIPT action 33
identifying studies 19
running VB.NET scripts 43

S

sample code
encrypting passwords 52
setting study UID 20
scheduled worklists 86
ScreenAttributeChange event 58
sending on startup 12
single sign-on 16
ScreenToCentreOn property 69
setting on startup 12
script example
add study to Image area 19, 34
build URL 35
clear Image area and activate List area 34
display search worklist 34
script interfaces 73–92
scripting 33
series attribute, standard worklists 84
SET_FOCUS 44
SET_WINDOW_STATE 44
setting window state 44
SHOW_SELECT 40
ShowDisplayList method 63

adding studies to a worklist 20
ShowDisplayListTrigger event 59
showing List and Text areas 40
sign off worklists..... 88
signing off of IMPAX and an integrated application.....17–18
signing onto IMPAX and an integrated application.....12–17
single sign-off 17–18
single sign-on.....12, 26
single-study dictation..... 28
size
buttons.....50, 67
font 50, 67
standard worklists 84–86
START_DICTATION action 30
starting dictation 28
multiple studies 29
two-state mode..... 28
starting dictation application 26
starting dictations..... 38
starting IMPAX Client 12
starting programs 65
STATUS Action
adding studies and priors to cycle list..... 37
clearing the Image area..... 38
STUDY_CHANGED 42
STATUS action
adding studies to cycle list 37
bringing a study forward 24–25
bringing studies forward..... 46
canceling dictation 31
clearing the Image area..... 23
updating a study status..... 47
status of studies 26
changing 31
setting 70
stopping dictation..... 31
studies
adding to a worklist.....20, 61
bringing forward 24
identifying.....19, 70
opening 21
setting accession number..... 64
study attributes
scheduled worklists 87–88
standard worklists 84, 86
study attributes, teaching files 89, 91
study status..... 26
changing 31
setting 70
Study Status property
ending dictation 28
StudyId property..... 70
adding studies to a worklist 61
identifying studies 20
opening studies 22
StudyIdentifierChange event 59
identifying studies 20
opening studies 22
StudyRead method..... 63
StudyReadTrigger event..... 59
StudyStatus property 70
canceling dictation..... 31
ending dictation..... 31
opening studies..... 22
starting and ending dictation 29
starting dictation..... 28, 30
starting multiple-study dictation 30
StudyStatusChange event..... 60
canceling dictation..... 31
ending dictation..... 31
opening studies..... 22
starting and ending dictation 29
starting dictation..... 30
starting multiple-study dictation 30
SYNC_DISPLAY..... 45
SYNC_DISPLAY action 24–25
synchronizing reporting applications..... 27

T

taking focus..... 44
taskbar, Windows..... 41
teaching file search results 89
Text area
hiding 21
text in user interface..... 66
three-state dictation mode..... 27–29
trainee dictation status..... 27
two-state dictation mode..... 27–28
typeface 50, 66

U

UPDATE_STUDY_STATUS 46
UPDATE_STUDY_STATUS action..... 31
URL building..... 35
user interface
button size 67
text 66
user login 62, 68, 71
user validation 52
UserName property..... 71
setting on startup 12
single sign-off..... 17–18
single sign-on..... 13–16
UserNameChange event 60
sending on startup 12
single sign-off..... 17–18
single sign-on..... 13–16

V

VB.NET scripts 33
Visual Studio..... 33

W

what is new 9
Windows taskbar 41
Wizard toolbar 33
worklist
    available attributes 84–91
    column customization 84
writing scripts 33